Numerical Recipes in C

The Art of Scientific Computing

William H. Press

Harvard-Smithsonian Center for Astrophysics

Brian P. Flannery

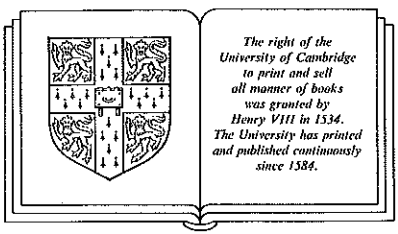
EXXON Research and Engineering Company

Saul A. Teukolsky

Department of Physics, Cornell University

William T. Vetterling

Polaroid Corporation



CAMBRIDGE UNIVERSITY PRESS
Cambridge
New York Port Chester
Melbourne Sydney

Chapter 13. Statistical Description of Data

13.0 Introduction

In this chapter and the next, the concept of *data* enters the discussion more prominently than before.

Data consist of numbers, of course. But these numbers are fed into the computer, not produced by it. These are numbers to be treated with considerable respect, never to be tampered with, nor subjected to a numerical process whose character you do not completely understand. You are well advised to acquire a reverence for data that is rather different from the "sporty" attitude which is sometimes allowable, or even commendable, in other numerical tasks.

The analysis of data inevitably involves some trafficking with the field of *statistics*, that gray area which is as surely not a branch of mathematics as it is neither a branch of science. In the following sections, you will repeatedly encounter the following paradigm:

- apply some formula to the data to compute "a statistic"
- compute where the value of that statistic falls in a probability distribution that is computed on the basis of some "null hypothesis"
- if it falls in a very unlikely spot, way out on a tail of the distribution, conclude that the null hypothesis is *false* for your data set

If a statistic falls in a *reasonable* part of the distribution, you must not make the mistake of concluding that the null hypothesis is "verified" or "proved." That is the curse of statistics, that it can never prove things, only disprove them! At best, you can substantiate a hypothesis by ruling out, statistically, a whole long list of competing hypotheses, every one that has ever been proposed. After a while your adversaries and competitors will give up trying to think of alternative hypotheses, or else they will grow old and die, and *then your hypothesis will become accepted.* Sounds crazy, we know, but that's how science works!

In this book we make a somewhat arbitrary distinction between data analysis procedures that are *model-independent* and those that are *model-dependent.* In the former category, we include so-called *descriptive statistics* that characterize a data set in general terms: its mean, variance, and so on. We also include statistical tests which seek to establish the "sameness" or "differentness" of two or more data sets, or which seek to establish and

measure a degree of *correlation* between two data sets. These subjects are discussed in this chapter.

In the other category, model-dependent statistics, we lump the whole subject of fitting data to a theory, parameter estimation, least-squares fits, and so on. Those subjects are introduced in Chapter 14.

Sections 13.1–13.3 of this chapter deal with so-called *measures of central tendency*, the moments of a distribution, the median and mode. In §13.4 we learn to test whether different data sets are drawn from distributions with different values of these measures of central tendency. This leads naturally, in §13.5 to the more general question of whether two distributions can be shown to be (significantly) different.

In §13.6–§13.8, we deal with *measures of association* for two distributions. We want to determine whether two variables are "correlated" or "dependent" on one another. If they are, we want to characterize the degree of correlation in some simple ways. The distinction between parametric and nonparametric (rank) methods is emphasized.

Section 13.9 touches briefly on the murky area of data smoothing.

This chapter draws mathematically on the material on special functions that was presented in Chapter 6, especially §6.1–§6.3. You may wish, at this point, to review those sections.

REFERENCES AND FURTHER READING:

Bevington, Philip R. 1969, *Data Reduction and Error Analysis for the Physical Sciences* (New York: McGraw-Hill).

Kendall, Maurice, and Stuart, Alan. 1977, *The Advanced Theory of Statistics*, 4th ed. (London: Griffin and Co.).

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill).

13.1 Moments of a Distribution: Mean, Variance, Skewness, and so forth

When a set of values has a sufficiently strong central tendency, that is, a tendency to cluster around some particular value, then it may be useful to characterize the set by a few numbers that are related to its *moments*, the sums of integer powers of the values.

Best known is the *mean* of the values $x_1, \ldots, x_N$,

$$\overline{x} = \frac{1}{N} \sum_{j=1}^{N} x_j \tag{13.1.1}$$

which estimates the value around which central clustering occurs. Note the use of an overbar to denote the mean; angle brackets are an equally common

notation, e.g. $\langle x \rangle$. You should be aware of the fact that the mean is not the only available estimator of this quantity, nor is it necessarily the best one. For values drawn from a probability distribution with very broad "tails," the mean may converge poorly, or not at all, as the number of sampled points is increased. Alternative estimators, the *median* and the *mode*, are discussed in §13.2 and §13.3.

Having characterized a distribution's central value, one conventionally next characterizes its "width" or "variability" around that value. Here again, more than one measure is available. Most common is the *variance*,

$$\mathrm{Var}(x_1 \dots x_N) = \frac{1}{N-1} \sum_{j=1}^{N} (x_j - \overline{x})^2 \tag{13.1.2}$$

or its square root, the *standard deviation*,

$$\sigma(x_1 \dots x_N) = \sqrt{\mathrm{Var}(x_1 \dots x_N)} \tag{13.1.3}$$

Equation (13.1.2) estimates the mean squared-deviation of x from its mean value. There is a long story about why the denominator of (13.1.2) is $N-1$ instead of N. If you have never heard that story, you may consult any good statistics text. Here we will be content to note that the $N-1$ *should* be changed to N if you are ever in the situation of measuring the variance of a distribution whose mean $\overline{x}$ is known *a priori* rather than being estimated from the data. (We might also comment that if the difference between N and $N-1$ ever matters to you, then you are probably up to no good anyway – e.g., trying to substantiate a questionable hypothesis with marginal data.)

As the mean depends on the first moment of the data, so do the variance and standard deviation depend on the second moment. It is not uncommon, in real life, to be dealing with a distribution whose second moment does not exist (i.e. is infinite). In this case, the variance or standard deviation is useless as a measure of the data's width around its central value: the values obtained from equations (13.1.2) or (13.1.3) will not converge with increased numbers of points, nor show any consistency from data set to data set drawn from the same distribution. This can occur even when the width of the peak looks, by eye, perfectly finite. A more robust estimator of the width is the *average deviation* or *mean absolute deviation*, defined by

$$\mathrm{ADev}(x_1 \dots x_N) = \frac{1}{N} \sum_{j=1}^{N} |x_j - \overline{x}| \tag{13.1.4}$$

Statisticians have historically sniffed at the use of (13.1.4) instead of (13.1.2), since the absolute value brackets in (13.1.4) are "nonanalytic" and make theorem-proving difficult. In recent years, however, the fashion has changed, and the subject of *robust estimation* (meaning, estimation for broad

distributions with significant numbers of "outlier" points) has become a popular and important one. Higher moments, or statistics involving higher powers of the input data, are almost always less robust than lower moments or statistics that involve only linear sums or (the lowest moment of all) counting.

That being the case, the *skewness* or *third moment*, and the *kurtosis* or *fourth moment* should be used with caution or, better yet, not at all.

The skewness characterizes the degree of asymmetry of a distribution around its mean. While the mean, standard deviation, and average deviation are *dimensional* quantities, that is, have the same units as the measured quantities x_j, the skewness is conventionally defined in such a way as to make it *nondimensional*. It is a pure number that characterizes only the shape of the distribution. The usual definition is

$$\text{Skew}(x_1 \dots x_N) = \frac{1}{N} \sum_{j=1}^{N} \left[\frac{x_j - \overline{x}}{\sigma} \right]^3 \tag{13.1.5}$$

where $\sigma = \sigma(x_1 \dots x_N)$ is the distribution's standard deviation (13.1.3). A positive value of skewness signifies a distribution with an asymmetric tail extending out towards more positive x; a negative value signifies a distribution whose tail extends out towards more negative x (see Figure 13.1.1).

Of course, any set of N measured values is likely to give a nonzero value for (13.1.5), even if the underlying distribution is in fact symmetrical (has zero skewness). For (13.1.5) to be meaningful, we need to have some idea of *its* standard deviation as an estimator of the skewness of the underlying distribution. Unfortunately, that depends on the shape of the underlying distribution, and rather critically on its tails! For the idealized case of a normal (Gaussian) distribution, the standard deviation of (13.1.5) is approximately $\sqrt{6/N}$. In real life it is good practice to believe in skewnesses only when they are several or many times as large as this.

The kurtosis is also a nondimensional quantity. It measures the relative peakedness or flatness of a distribution. Relative to what? A normal distribution, what else! A distribution with positive kurtosis is termed *leptokurtic*; the outline of the Matterhorn is an example. A distribution with negative kurtosis is termed *platykurtic*; the outline of a loaf of bread is an example. (See Figure 13.1.1.) And, you will no doubt be pleased to hear, an in-between distribution is termed *mesokurtic*.

The conventional definition of the kurtosis is

$$\text{Kurt}(x_1 \dots x_N) = \left\{ \frac{1}{N} \sum_{j=1}^{N} \left[\frac{x_j - \overline{x}}{\sigma} \right]^4 \right\} - 3 \tag{13.1.6}$$

where the -3 term makes the value zero for a normal distribution.

The standard deviation of (13.1.6) as an estimator of the kurtosis of an underlying normal distribution is $\sqrt{24/N}$. However, the kurtosis depends on

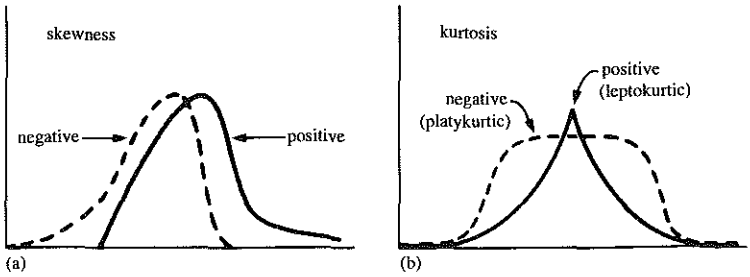


Figure 13.1.1. Distributions whose third and fourth moments are significantly different from a normal (Gaussian) distribution. (a) Skewness or third moment. (b) Kurtosis or fourth moment.

such a high moment that there are many real-life distributions for which the standard deviation of (13.1.6) as an estimator is effectively infinite.

Calculation of the quantities defined in this section is perfectly straightforward. Many textbooks use the binomial theorem to expand out the definitions into sums of various powers of the data, e.g. the familiar

$$\mathrm{Var}(x_1 \ldots x_N) = \frac{1}{N-1}\left[\left(\sum_{j=1}^{N} x_j^2\right) - N\overline{x}^2\right] \approx \overline{x^2} - \overline{x}^2 \qquad (13.1.7)$$

but this is generally unjustifiable in terms of computing speed and/or roundoff error.

```
#include <math.h>

void moment(data,n,ave,adev,sdev,svar,skew,curt)
int n;
float data[],*ave,*adev,*sdev,*svar,*skew,*curt;
Given an array data[1..n], this routine returns its mean ave, average deviation adev, standard
deviation sdev, variance var, skewness skew, and kurtosis curt.
{
    int j;
    float s,p;
    void nrerror();

    if (n <= 1) nrerror("n must be at least 2 in MOMENT");
    s=0.0;                                  First pass to get the mean.
    for (j=1;j<=n;j++) s += data[j];
    *ave=s/n;
    *adev=(*svar)=(*skew)=(*curt)=0.0;      Second pass to get the first (absolute), second,
    for (j=1;j<=n;j++) {                        third, and fourth moments of the deviation
        *adev += fabs(s=data[j]-(*ave));        from the mean.
        *svar += (p=s*s);
        *skew += (p *= s);
        *curt += (p *= s);
    }
    *adev /= n;                             Put the pieces together according to the conven-
    *svar /= (n-1);                             tional definitions.
    *sdev=sqrt(*svar);
    if (*svar) {
        *skew /= (n*(*svar)*(*sdev));
```

```
        *curt=(*curt)/(n*(*svar)*(*svar))-3.0;
    } else nrerror("No skew/kurtosis when variance = 0 (in MOMENT)");
}
```

REFERENCES AND FURTHER READING:

Downie, N.M., and Heath, R.W. 1965, *Basic Statistical Methods*, 2nd ed. (New York: Harper and Row), Chapters 4 and 5.

Bevington, Philip R. 1969, *Data Reduction and Error Analysis for the Physical Sciences* (New York: McGraw-Hill), Chapter 2.

Kendall, Maurice, and Stuart, Alan. 1977, *The Advanced Theory of Statistics*, 4th ed. (London: Griffin and Co.), vol. 1, §10.15

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill), §14.1

13.2 Efficient Search for the Median

The median of a probability distribution function $p(x)$ is the value x_{med} for which larger and smaller values of x are equally probable:

$$\int_{-\infty}^{x_{med}} p(x)\,dx = \frac{1}{2} = \int_{x_{med}}^{\infty} p(x)\,dx \tag{13.2.1}$$

The median of a distribution is estimated from a sample of values $x_1, \ldots, x_N$ by finding that value x_i which has equal numbers of values above it and below it. Of course, this is not possible when N is even. In that case it is conventional to estimate the median as the mean of the unique *two* central values. If the values x_j $j = 1, \ldots, N$ are sorted into ascending (or, for that matter, descending) order, then the formula for the median is

$$\begin{aligned} x_{med} &= x_{(N+1)/2} \qquad N \text{ odd} \\ &= \frac{1}{2}(x_{N/2} + x_{(N/2)+1}) \quad N \text{ even} \end{aligned} \tag{13.2.2}$$

If a distribution has a strong central tendency, so that most of its area is under a single peak, then the median is an estimator of the central value. It is a more robust estimator than the mean is: the median fails as an estimator only if the area in the tails is large, while the mean fails if the first moment of the tails is large; it is easy to construct examples where the first moment of the tails is large even though their area is negligible.

To find the median of a set of values, one can proceed by sorting the set and then applying (13.2.2). This is a process of order $N \log N$. You might think that this is wasteful, since it provides much more information

than just the median (e.g. the upper and lower quartile points, the deciles, etc.). In fact, there are faster methods known for finding the median only, in (e.g.) order $N \log\log N$ or $N^{2/3} \log N$ operations. For details, see Knuth. As a practical matter, these differences in speed are not usually substantial enough to be important. All of the "combinatorial" methods share the same disadvantage: they rearrange all the data, either in place or else in additional scratch memory.

```
void mdian1(x,n,xmed)
float x[],*xmed;
int n;
Given an array x[1..n], returns their median value xmed. The array x is modified and returned
sorted into ascending order.
{
    int n2,n2p;
    void sort();

    sort(n,x);                          This routine is in §8.2.
    n2p=(n2=n/2)+1;
    *xmed=(n % 2 ? x[n2p] : 0.5*(x[n2]+x[n2p]));
}
```

There are times, however, when it is quite inconvenient to rearrange all the data just to find the median value. For example, you might have a very long tape of values. While you may be willing to make a number of consecutive passes through the tape, sorting the whole thing is a task of considerably greater magnitude, requiring an additional external storage device, etc. The nice thing about the *mean* is that it can be computed in one pass through the data. Can we do anything like that for the median?

Yes, actually, if you are willing to make on the order of $\log N$ passes through the data. The method for doing so does not seem to be generally known, so we will explain it in some detail: The median (in the case of even N, say) satisfies the equation

$$\sum_{j=1}^{N} \frac{x_j - x_{med}}{|x_j - x_{med}|} = 0 \tag{13.2.3}$$

since each term in the sum is ± 1, depending on whether x_j is larger or smaller than the median value. Equation (13.2.3) can be rewritten as

$$x_{med} = \frac{\sum_{j=1}^{N} \frac{x_j}{|x_j - x_{med}|}}{\sum_{j=1}^{N} \frac{1}{|x_j - x_{med}|}} \tag{13.2.4}$$

which shows that the median is a kind of weighted average of the points, weighted by the reciprocal of their distance from the median.

Equation (13.2.4) is an implicit equation for x_{med}. It can be solved iteratively, using the previous guess on the right-hand side to compute the

next guess. With this procedure (supplemented by a couple of tricks explained in comments below), convergence is obtained in of order $\log N$ passes through the data. The following routine pretends that the data array is in memory, but can easily be modified to make sequential passes through an external tape.

```
#include <math.h>

#define BIG 1.0e30
#define AFAC 1.5
#define AMP 1.5
```

Here, AMP is an overconvergence factor: on each iteration, we move the guess by this factor more than (13.2.4) would naively indicate. AFAC is a factor used to optimize the size of the "smoothing constant" eps at each iteration.

```
void mdian2(x,n,xmed)
int n;
float x[],*xmed;
```

Given an array x[1..n], returns their median value xmed. The array x is not modified, and is accessed sequentially in each consecutive pass.

```
{
    int np,nm,j;
    float xx,xp,xm,sumx,sum,eps,stemp,dum,ap,am,aa,a;

    a=0.5*(x[1]+x[n]);                  This can be any first guess for the median.
    eps=fabs(x[n]-x[1]);
```

This can be any first guess for the characteristic spacing of the data points near the median.

```
    am = -(ap=BIG);                     ap and am are upper and lower bounds on the median.
    for (;;) {
        sum=sumx=0.0;                   Here we start one pass through the data.
        np=nm=0;                        Number of points above the current guess, and below it.
        xm = -(xp=BIG);
```

Value of the point above and closest to the guess, and below and closest.

```
        for (j=1;j<=n;j++) {            Go through the points,
            xx=x[j];
            if (xx != a) {              omit a zero denominator in the sums,
                if (xx > a) {               update the diagnostics,
                    ++np;
                    if (xx < xp) xp=xx;
                } else if (xx < a) {
                    ++nm;
                    if (xx > xm) xm=xx;
                }
                sum += dum=1.0/(eps+fabs(xx-a));    The smoothing constant is used here.
                sumx += xx*dum;                     accumulate the sums.
            }
        }
        stemp=(sumx/sum)-a;
        if (np-nm >= 2) {               Guess is too low; make another pass,
            am=a;                       with a new lower bound,
            aa = stemp < 0.0 ? xp : xp+stemp*AMP;       a new best guess
            if (aa > ap) aa=0.5*(a+ap);     (but no larger than the upper bound)
            eps=AFAC*fabs(aa-a);            and a new smoothing factor.
            a=aa;
        } else if (nm-np >= 2) {        Guess is too high; make another pass,
            ap=a;                       with a new upper bound,
            aa = stemp > 0.0 ? xm : xm+stemp*AMP;       a new best guess
            if (aa < am) aa=0.5*(a+am);     (but no smaller than the lower bound)
            eps=AFAC*fabs(aa-a);            and a new smoothing factor.
            a=aa;
        } else {                        Got it!
            if (n % 2 == 0) {           For even n median is always an average.
                *xmed = 0.5*(np == nm ? xp+xm : np > nm ? a+xp : xm+a);
```

```
        } else {                    For odd n median is always one point.
            *xmed = np == nm ? a : np > nm ? xp : xm;
        }
        return;
    }
  }
}
```

It is typical for `mdian2` to take about 12 ± 2 passes for 10,000 points, and (presumably) about double this number for 100,000,000 points. One nice thing is that you can quit after a smaller number of passes if `np` and `nm` are close enough for your purposes. For example, it will generally take only about 6 passes (independent of the number of points) for them to be within a percent or so of each other, putting the guess `a` in the 50 ± 0.5^{th} percentile, close enough for Government work. Note also the possibility of doing the "endgame" by (on one pass) copying to a relatively small array only those points between the bounds `ap` and `am`, along with the counts of how many points are above and below these bounds.

REFERENCES AND FURTHER READING:

Knuth, Donald E. 1973, *Sorting and Searching*, vol. 3 of *The Art of Computer Programming* (Reading, Mass.: Addison-Wesley), pp. 216ff.

13.3 Estimation of the Mode for Continuous Data

The *mode* of a probability distribution function $p(x)$ is the value of x where it takes on a maximum value. The mode is useful primarily when there is a single, sharp maximum, in which case it estimates the central value. Occasionally, a distribution will be *bimodal*, with two relative maxima; then one may wish to know the two modes individually. Note that, in such cases, the mean and median are not very useful, since they may give only a "compromise" value between the two peaks.

Elementary statistics texts seem to have the notion that the only way to estimate the mode is to bin the data into a *histogram* or bar-graph, selecting as the estimated value the midpoint of the bin with the largest number of points. If the data are intrinsically integer-valued, then the binning is automatic, and in some sense "natural," so this procedure is not unreasonable.

Real-life data are more often not integer-valued, however, but real-valued. (True, data stored with finite precision can be viewed as integer valued; but the number of possible integers is a number so large, e.g. 2^{32}, that you will rarely have more than one data point per "integer" bin.) It is almost always a bad idea to bin continuous data, since the binning throws away information.

The right way to estimate the mode of a distribution from a set of sampled points goes by the name "estimating the rate of an inhomogeneous Poisson process by J^{th} waiting times":

- First, sort all the points x_j $j = 1, \ldots, N$ into ascending order.
- Second, select an integer J as your "window size," the resolution (measured in number of consecutive points) over which the distribution function will be smeared. Smaller J gives better resolution in x and the chance of finding a high, but very narrow peak; but smaller J gives poorer accuracy in finding the true maximum as opposed to a chance fluctuation in the data. We will discuss this below. J should never be smaller than 3 and should be as large as you can tolerate.
- Third, for every $i = 1, \ldots, N - J$, estimate $p(x)$ as follows:

$$p\left(\tfrac{1}{2}[x_i + x_{i+J}]\right) \approx \frac{J}{N(x_{i+J} - x_i)} \tag{13.3.1}$$

- Fourth, take the value $\frac{1}{2}(x_i + x_{i+J})$ of the largest of these estimates to be the estimated mode.

The standard deviation of (13.3.1) as an estimator of $p(x)$ is

$$\sigma\left[p\left(\tfrac{1}{2}[x_i + x_{i+J}]\right)\right] \approx \frac{\sqrt{J}}{N(x_{i+J} - x_i)} \tag{13.3.2}$$

In other words, the fractional error is on the order of $1/\sqrt{J}$. So, the selection between two possible candidates for the mode is not significant unless their p's differ by several times (13.3.2).

A more subtle problem is how to compare mode candidate values obtained with different values of J. This problem occurs in practice quite often: the data may have a narrow peak containing only a few points, which, for larger values of J, "washes out" in favor of a broader peak at some different value of x. We know of no rigorous solution to this problem, but we do know at least a high-brow heuristic technique:

Suppose we have some set of mode candidates with different J's. Let x_j be the value of the mode for the case $J = j$, and let p_j be the estimate of $p(x)$ at that point. Let Δx_j denote the range of the window around candidate x_j; for example, it might be $\Delta x_7 = x_{19} - x_{12}$, if the mode candidate with window length 7 occurred between points 12 and 19.

Let H_j denote the hypothesis "the true mode is at x_j, while all the other mode candidates are chance fluctuations of a distribution function no larger than the true mode." If H_j is true, then we need to know how unlikely each of the other Δx_n's are, in other words, how likely is it that the range around

- First, sort all the points x_j $j = 1, \ldots, N$ into ascending order.
- Second, select an integer J as your "window size," the resolution (measured in number of consecutive points) over which the distribution function will be smeared. Smaller J gives better resolution in x and the chance of finding a high, but very narrow peak; but smaller J gives poorer accuracy in finding the true maximum as opposed to a chance fluctuation in the data. We will discuss this below. J should never be smaller than 3 and should be as large as you can tolerate.
- Third, for every $i = 1, \ldots, N - J$, estimate $p(x)$ as follows:

$$p\left(\tfrac{1}{2}[x_i + x_{i+J}]\right) \approx \frac{J}{N(x_{i+J} - x_i)} \tag{13.3.1}$$

- Fourth, take the value $\frac{1}{2}(x_i + x_{i+J})$ of the largest of these estimates to be the estimated mode.

The standard deviation of (13.3.1) as an estimator of $p(x)$ is

$$\sigma\left[p\left(\tfrac{1}{2}[x_i + x_{i+J}]\right)\right] \approx \frac{\sqrt{J}}{N(x_{i+J} - x_i)} \tag{13.3.2}$$

In other words, the fractional error is on the order of $1/\sqrt{J}$. So, the selection between two possible candidates for the mode is not significant unless their p's differ by several times (13.3.2).

A more subtle problem is how to compare mode candidate values obtained with different values of J. This problem occurs in practice quite often: the data may have a narrow peak containing only a few points, which, for larger values of J, "washes out" in favor of a broader peak at some different value of x. We know of no rigorous solution to this problem, but we do know at least a high-brow heuristic technique:

Suppose we have some set of mode candidates with different J's. Let x_j be the value of the mode for the case $J = j$, and let p_j be the estimate of $p(x)$ at that point. Let Δx_j denote the range of the window around candidate x_j; for example, it might be $\Delta x_7 = x_{19} - x_{12}$, if the mode candidate with window length 7 occurred between points 12 and 19.

Let H_j denote the hypothesis "the true mode is at x_j, while all the other mode candidates are chance fluctuations of a distribution function no larger than the true mode." If H_j is true, then we need to know how unlikely each of the other Δx_n's are, in other words, how likely is it that the range around x_n should be as short or shorter than observed if the true probability density is p_j rather than p_n? This is given by the gamma probability distribution

previously introduced in §7.3,

$$\begin{aligned}\int_0^{\Delta x_n} (Np_j)\frac{(Np_j x)^{n-1}}{(n-1)!}e^{-Np_j x}dx &= P(n, Np_j\Delta x_n) \\ &= P(n, \frac{np_j}{p_n}) \\ &= P(n, \frac{j\Delta x_n}{\Delta x_j})\end{aligned} \tag{13.3.3}$$

In this equation, $P(a,x)$ is the incomplete gamma function (see equation 6.2.1), and the last two equalities follow from equation (13.3.1).

The *likelihood* of hypothesis H_j is now the product of these probabilities over all the *other* n's, not including j:

$$\text{Likelihood}(H_j) = \prod_{n \neq j} P(n, \frac{j\Delta x_n}{\Delta x_j}) \tag{13.3.4}$$

You select the hypothesis H_j which has the the (so-called) *maximum likelihood*. This method is straightforward to program, using the incomplete gamma function routine `gammp` in §6.2.

REFERENCES AND FURTHER READING:

Parzen, Emanual, 1962, *Stochastic Processes* (San Francisco: Holden Day).

13.4 Do Two Distributions Have the Same Means or Variances?

Not uncommonly we want to know whether two distributions have the same mean. For example, a first set of measured values may have been gathered before some event, a second set after it. We want to know whether the event, a "treatment" or a "change in a control parameter," made a difference.

Our first thought is to ask "how many standard deviations" one sample mean is from the other. That number may in fact be a useful thing to know. It does relate to the strength or "importance" of a difference of means *if that difference is genuine*. However, by itself, it says nothing about whether the difference *is* genuine, that is, statistically significant. A difference of means can be very small compared to the standard deviation, and yet very significant, if the number of data points is large. Conversely, a difference may be moderately large but not significant, if the data are sparse. We will be

meeting these distinct concepts of *strength* and *significance* several times in the next few sections.

A quantity that measures the significance of a difference of means is not the number of standard deviations that they are apart, but the number of so-called *standard errors* that they are apart. The standard error of a set of values is their standard deviation divided by the square root of their number; the standard error estimates the standard deviation of the sample mean as an estimator of the population (or "true") mean.

Student's t-test for Significantly Different Means

Applying the concept of standard error, the conventional statistic for measuring the significance of a difference of means is termed *Student's t*. When the two distributions are thought to have the same variance, but possibly different means, then Student's t is computed as follows: First, estimate the standard error of the difference of the means, s_D, from the "pooled variance" by the formula

$$s_D = \sqrt{\frac{\sum_{one}(x_i - \overline{x_{one}})^2 + \sum_{two}(x_i - \overline{x_{two}})^2}{N_1 + N_2 - 2}\left(\frac{1}{N_1} + \frac{1}{N_2}\right)} \tag{13.4.1}$$

where each sum is over the points in one sample, the first or second, each mean likewise refers to one sample or the other, and N_1 and N_2 are the numbers of points in the first and second samples respectively. Second, compute t by

$$t = \frac{\overline{x_{one}} - \overline{x_{two}}}{s_D} \tag{13.4.2}$$

Third, evaluate the significance of this value of t for Student's distribution with $N_1 + N_2 - 2$ degrees of freedom, by equations (6.3.7) and (6.3.9), and by the routine `betai` (incomplete beta function) of §6.3.

The significance is a number between zero and one, and is the probability that $|t|$ could be this large or larger just by chance, for distributions with equal means. Therefore, a small numerical value of the significance (0.05 or 0.01) means that the observed difference is "very significant." The function $A(t|\nu)$ in equation (6.3.7) is one minus the significance.

As a routine, we have

```
#include <math.h>

void ttest(data1,n1,data2,n2,t,prob)
int n1,n2;
float data1[],data2[],*t,*prob;
Given the arrays data1[1..n1] and data2[1..n2], this routine returns Student's t as t, and
its significance as prob, small values of prob indicating that the arrays have significantly
different means. The data arrays are assumed to be drawn from populations with the same
true variance.
{
    float var1,var2,svar,df,ave1,ave2;
    void avevar();
    float betai();

    avevar(data1,n1,&ave1,&var1);
    avevar(data2,n2,&ave2,&var2);
    df=n1+n2-2;                                   Degrees of freedom.
    svar=((n1-1)*var1+(n2-1)*var2)/df;            Pooled variance.
    *t=(ave1-ave2)/sqrt(svar*(1.0/n1+1.0/n2));
    *prob=betai(0.5*df,0.5,df/(df+(*t)*(*t)));    See equation (6.3.9).
}
```

which makes use of the following routine for computing the mean and variance of a set of numbers,

```
void avevar(data,n,ave,svar)
float data[],*ave,*svar;
int n;
Given array data[1..n], returns its mean as ave and its variance as var.
{
    int j;
    float s;

    *ave=(*svar)=0.0;
    for (j=1;j<=n;j++) *ave += data[j];
    *ave /= n;
    for (j=1;j<=n;j++) {
        s=data[j]-(*ave);
        *svar += s*s;
    }
    *svar /= (n-1);
}
```

The next case to consider is where the two distributions have significantly different variances, but we nevertheless want to know if their means are the same or different. (A treatment for baldness has caused some patients to *lose* all their hair and turned others into werewolves, but we want to know if it helps cure baldness *on the average*!) Be suspicious of the unequal-variance t-test: if two distributions have very different variances, then they may also be substantially different in shape; in that case, the difference of the means may not be a particularly useful thing to know.

To find out whether the two data sets have variances that are significantly different, you use the *F-test*, described later on in this section.

The relevant statistic for the unequal variance t-test is

$$t = \frac{\overline{x_{one}} - \overline{x_{two}}}{[\mathrm{Var}(x_{one})/N_1 + \mathrm{Var}(x_{two})/N_2]^{1/2}} \tag{13.4.3}$$

This statistic is distributed *approximately* as Student's t with a number of degrees of freedom equal to

$$\frac{\left[\dfrac{\mathrm{Var}(x_{one})}{N_1} + \dfrac{\mathrm{Var}(x_{two})}{N_2}\right]^2}{\dfrac{[\mathrm{Var}(x_{one})/N_1]^2}{N_1 - 1} + \dfrac{[\mathrm{Var}(x_{two})/N_2]^2}{N_2 - 1}} \tag{13.4.4}$$

Expression (13.4.4) is in general not an integer, but equation (6.3.7) doesn't care.

The routine is

```
#include <math.h>

static float sqrarg;
#define SQR(a) (sqrarg=(a),sqrarg*sqrarg)

void tutest(data1,n1,data2,n2,t,prob)
float data1[],data2[],*t,*prob;
int n1,n2;
Given the arrays data1[1..n1] and data2[1..n2], this routine returns Student's t as t, and its
significance as prob, small values of prob indicating that the arrays have significantly different
means. The data arrays are allowed to be drawn from populations with unequal variances.
{
    float var1,var2,df,ave1,ave2;
    void avevar();
    float betai();

    avevar(data1,n1,&ave1,&var1);
    avevar(data2,n2,&ave2,&var2);
    *t=(ave1-ave2)/sqrt(var1/n1+var2/n2);
    df=SQR(var1/n1+var2/n2)/(SQR(var1/n1)/(n1-1)+SQR(var2/n2)/(n2-1));
    *prob=betai(0.5*df,0.5,df/(df+SQR(*t)));
}
```

Our final example of a Student's t test is the case of *paired samples*. Here we imagine that much of the variance in *both* samples is due to effects which are point-by-point identical in the two samples. For example, we might have two job candidates who have each been rated by the same ten members of a hiring committee. We want to know if the means of the ten scores differ significantly. We first try `ttest` above, and obtain a value of `prob` which is not especially significant (e.g., > 0.05). But perhaps the significance is being washed out by the tendency of some committee members always to give high scores, others always to give low scores, which increases the apparent variance

and thus decreases the significance of any difference in the means. We thus try the paired-sample formulas,

$$\mathrm{Cov}(x_{one}, x_{two}) \equiv \frac{1}{N-1}\sum_{i=1}^{N}(x_{one\,i} - \overline{x_{one}})(x_{two\,i} - \overline{x_{two}}) \tag{13.4.5}$$

$$s_D = \sqrt{\frac{\mathrm{Var}(x_{one}) + \mathrm{Var}(x_{two}) - 2\mathrm{Cov}(x_{one}, x_{two})}{N}} \tag{13.4.6}$$

$$t = \frac{\overline{x_{one}} - \overline{x_{two}}}{s_D} \tag{13.4.7}$$

where N is the number in each sample (number of pairs). Notice that it is important that a particular value of i label the corresponding points in each sample, that is, the ones that are paired. The significance of the t statistic in (13.4.5) is evaluated for $N - 1$ degrees of freedom.

The routine is

```
#include <math.h>

void tptest(data1,data2,n,t,prob)
float data1[],data2[],*t,*prob;
int n;
Given the paired arrays data1[1..n] and data2[1..n], this routine returns Student's t for
paired data as t, and its significance as prob, small values of prob indicating a significant
difference of means.
{
    int j;
    float var1,var2,ave1,ave2,sd,df,cov=0.0;
    void avevar();
    float betai();

    avevar(data1,n,&ave1,&var1);
    avevar(data2,n,&ave2,&var2);
    for (j=1;j<=n;j++)
        cov += (data1[j]-ave1)*(data2[j]-ave2);
    cov /= df=n-1;
    sd=sqrt((var1+var2-2.0*cov)/n);
    *t=(ave1-ave2)/sd;
    *prob=betai(0.5*df,0.5,df/(df+(*t)*(*t)));
}
```

F-Test for Significantly Different Variances

The *F-test* tests the hypothesis that two samples have different variances by trying to reject the null hypothesis that their variances are actually consistent. The statistic F is the ratio of one variance to the other, so values either $\gg 1$ or $\ll 1$ will indicate very significant differences. The distribution of F in the null case is given in equation (6.3.11), which is evaluated using the routine `betai`. In the most common case, we are willing to disprove the null hypothesis (of equal variances) by either very large or very small values of F, so the correct significance is *two-tailed*, the sum of two incomplete beta functions. It turns out, by equation (6.3.3), that the two tails are always equal; we need only compute one, and double it. Occasionally, when the null hypothesis is strongly viable, the identity of the two tails can become confused, giving an indicated probability greater than one. Changing the probability to two minus itself correctly exchanges the tails. These considerations and equation (6.3.3) give the routine

```
void ftest(data1,n1,data2,n2,f,prob)
float data1[],data2[],*f,*prob;
int n1,n2;
Given the arrays data1[1..n1] and data2[1..n2], this routine returns the value of f, and
its significance as prob. Small values of prob indicate that the two arrays have significantly
different variances.
{
    float var1,var2,ave1,ave2,df1,df2;
    void avevar();
    float betai();

    avevar(data1,n1,&ave1,&var1);
    avevar(data2,n2,&ave2,&var2);
    if (var1 > var2) {          Make F the ratio of the larger variance to the smaller one.
        *f=var1/var2;
        df1=n1-1;
        df2=n2-1;
    } else {
        *f=var2/var1;
        df1=n2-1;
        df2=n1-1;
    }
    *prob = 2.0*betai(0.5*df2,0.5*df1,df2/(df2+df1*(*f)));
    if (*prob > 1.0) *prob=2.0-*prob;
}
```

REFERENCES AND FURTHER READING:

von Mises, Richard. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), Chapter IX(B).

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill), §17.2.

13.5 Are Two Distributions Different?

Given two sets of data, we can generalize the questions asked in the previous section and ask the single question: Are the two sets drawn from the same distribution function, or from different distribution functions? Equivalently, in proper statistical language, "Can we disprove, to a certain required level of significance, the null hypothesis that two data sets are drawn from the same population distribution function?" Disproving the null hypothesis in effect proves that the data sets are from different distributions. Failing to disprove the null hypothesis, on the other hand, only shows that the data sets can be *consistent* with a single distribution function. One can never *prove* that two data sets come from a single distribution, since (e.g.) no practical amount of data can distinguish between two distributions which differ only by one part in 10^{10}.

Proving that two distributions are different, or showing that they are consistent, is a task that comes up all the time in many areas of research: Are the visible stars distributed uniformly in the sky? (That is, is the distribution of stars as a function of declination — position in the sky — the same as the distribution of sky area as a function of declination?) Are educational patterns the same in Brooklyn as in the Bronx? (That is, are the distributions of people as a function of last-grade-attended the same?) Do two brands of fluorescent lights have the same distribution of burn-out times? Is the incidence of chicken pox the same for first-born, second-born, third-born children, etc.?

These four examples illustrate the four combinations arising from two different dichotomies: (1) Either the data is continuous or binned. (2) Either we wish to compare one data set to a known distribution, or we wish to compare two equally unknown data sets. The data sets on fluorescent lights and on stars are continuous, since we can be given lists of individual burnout times or of stellar positions. The data sets on chicken pox and educational level are binned, since we are given tables of numbers of events in discrete categories: first-born, second-born, etc.; or 6th Grade, 7th Grade, etc. Stars and chicken pox, on the other hand, share the property that the null hypothesis is a known distribution (distribution of area in the sky, or incidence of chicken pox in the general population). Fluorescent lights and educational level involve the comparison of two equally unknown data sets (the two brands, or Brooklyn and the Bronx).

One can always turn continuous data into binned data, by grouping the events into specified ranges of the continuous variable(s): Declinations between 0 and 10 degrees, 10 and 20, 20 and 30, etc. Binning involves a loss of information, however. Also, there is often considerable arbitrariness as to how the bins should be chosen. Along with many other investigators, we prefer to avoid unnecessary binning of data.

The accepted test for differences between binned distributions is the *chi-square test*. For continuous data as a function of a single variable, the most generally accepted test is the *Kolmogorov-Smirnov test*. We consider each in turn.

Chi-Square Test

Suppose that N_i is the number of events observed in the ith bin, and that n_i is the number expected according to some known distribution. Note that the N_i's are integers, while the n_i's may not be. Then the chi-square statistic is

$$\chi^2 = \sum_i \frac{(N_i - n_i)^2}{n_i} \tag{13.5.1}$$

where the sum is over all bins. A large value of χ^2 indicates that the null hypothesis (that the N_i's are drawn from the population represented by the n_i's) is rather unlikely.

Any term j in (13.5.1) with $0 = n_j = N_j$ should be omitted from the sum. A term with $n_j = 0$, $N_j \neq 0$ gives an infinite χ^2, as it should, since in this case the N_i's cannot possibly be drawn from the n_i's!

The *chi-square probability function* $Q(\chi^2|\nu)$ is an incomplete gamma function, and was already discussed in §6.2 (see equation 6.2.18). Strictly speaking $Q(\chi^2|\nu)$ is the probability that the sum of the squares of ν random *normal* variables of unit variance will be greater than χ^2. The terms in the sum (13.5.1) are not individually normal. However, if either the number of bins is large ($\gg 1$), or the number of events in each bin is large ($\gg 1$), then the chi-square probability function is a good approximation to the distribution of (13.5.1) in the case of the null hypothesis. Its use to estimate the significance of the chi-square test is standard.

The appropriate value of ν, the number of degrees of freedom, bears some additional discussion. If the data were collected in such a way that the n_i's were all determined in advance, and there were no *a priori* constraints on any of the N_i's, then ν equals the number of bins N_B (note that this is *not* the total number of *events*!). Much more commonly, the n_i's are normalized after the fact so that their sum equals the sum of the N_i's, the total number of events measured. In this case the correct value for ν is $N_B - 1$. If the model that gives the n_i's had additional free parameters that were adjusted after the fact to agree with the data, then each of these additional "fitted" parameters reduces ν by one additional unit. The number of these additional fitted parameters (*not* including the normalization of the n_i's) is commonly called the "number of constraints," so the number of degrees of freedom is $\nu = N_B - 1$ when there are "zero constraints."

We have, then, the following program:

```
void chsone(bins,ebins,nbins,knstrn,df,chsq,prob)
float bins[],ebins[],*df,*chsq,*prob;
int nbins,knstrn;
```

Given the array **bins[1..nbins]**, containing the observed numbers of events, and an array **ebins[1..nbins]** containing the expected numbers of events, and given the number of constraints **knstrn** (normally zero), this routine returns (trivially) the number of degrees of freedom **df**, and (nontrivially) the chi-square **chsq** and the significance **prob**. A small value

of prob indicates a significant difference between the distributions bins and ebins. Note that bins and ebins are both real arrays, although bins will normally contain integer values.

```
{
    int j;
    float temp;
    float gammq();
    void nrerror();

    *df=nbins-1-knstrn;
    *chsq=0.0;
    for (j=1;j<=nbins;j++) {
        if (ebins[j] <= 0.0) nrerror("Bad expected number in CHSONE");
        temp=bins[j]-ebins[j];
        *chsq += temp*temp/ebins[j];
    }
    *prob=gammq(0.5*(*df),0.5*(*chsq));          Chi-square probability function. See §6.2.
}
```

Next we consider the case of comparing *two* binned data sets. Let R_i be the number of events in bin i for the first data set, S_i the number of events in the same bin i for the second data set. Then the chi-square statistic is

$$\chi^2 = \sum_i \frac{(R_i - S_i)^2}{R_i + S_i} \tag{13.5.2}$$

Comparing (13.5.2) to (13.5.1), you should note that the denominator of (13.5.2) is *not* just the average of R_i and S_i (which would be an estimator of n_i in 13.5.1). Rather, it is twice the average, the sum. The reason is that each term in a chi-square sum is supposed to approximate the square of a normally distributed quantity with unit variance. The variance of the difference of two normal quantities is the sum of their individual variances, not the average.

If the data were collected in such a way that the sum of the R_i's is necessarily equal to the sum of S_i's, then the number of degrees of freedom is equal to one less than the number of bins, $N_B - 1$ (that is, knstrn=0), the usual case. If this requirement were absent, then the number of degrees of freedom would be N_B. Example: A birdwatcher wants to know whether the distribution of sighted birds as a function of species is the same this year as last. Each bin corresponds to one species. If the birdwatcher takes his data to be the first 1000 birds that he saw in each year, then the number of degrees of freedom is $N_B - 1$. If he takes his data to be all the birds he saw on a random sample of days, the same days in each year, then the number of degrees of freedom is N_B (knstrn= -1). In this latter case, note that he is also testing whether the birds were more numerous overall in one year or the other: that is the extra degree of freedom. Of course, any additional constraints on the data set lower the number of degrees of freedom (i.e., increase knstrn to *positive* values) in accordance with their number.

The program is

```
void chstwo(bins1,bins2,nbins,knstrn,df,chsq,prob)
float bins1[],bins2[],*df,*chsq,*prob;
int nbins,knstrn;
```

Given the arrays **bins1**[1..**nbins**] and **bins2**[1..**nbins**], containing two sets of binned data, and given the number of additional constraints **knstrn** (normally 0 or −1), this routine returns the number of degrees of freedom **df**, the chi-square **chsq** and the significance **prob**. A small value of **prob** indicates a significant difference between the distributions **bins1** and **bins2**. Note that **bins1** and **bins2** are both real arrays, although they will normally contain integer values.

```
{
    int j;
    float temp;
    float gammq();

    *df=nbins-1-knstrn;
    *chsq=0.0;
    for (j=1;j<=nbins;j++)
        if (bins1[j] == 0.0 && bins2[j] == 0.0)
            *df -= 1.0;                          No data means one less degree of freedom.
        else {
            temp=bins1[j]-bins2[j];
            *chsq += temp*temp/(bins1[j]+bins2[j]);
        }
    *prob=gammq(0.5*(*df),0.5*(*chsq));          Chi-square probability function. See §6.2.
}
```

Kolmogorov-Smirnov Test

The Kolmogorov-Smirnov (or K–S) test is applicable to unbinned distributions that are functions of a single independent variable, that is, to data sets where each data point can be associated with a single number (lifetime of each lightbulb when it burns out, or declination of each star). In such cases, the list of data points can be easily converted to an unbiased estimator $S_N(x)$ of the *cumulative* distribution function of the probability distribution from which it was drawn: If the N events are located at values x_i, $i = 1, \ldots, N$, then $S_N(x)$ is the function giving the fraction of data points to the left of a given value x. This function is obviously constant between consecutive (i.e. sorted into ascending order) x_i's, and jumps by the same constant $1/N$ at each x_i. (See Figure 13.5.1).

Different distribution functions, or sets of data, give different cumulative distribution function estimates by the above procedure. However, all cumulative distribution functions agree at the smallest allowable value of x (where they are zero), and at the largest allowable value of x (where they are unity). (The smallest and largest values might of course be $\pm\infty$.) So it is the behavior between the largest and smallest values that distinguishes distributions.

One can think of any number of statistics to measure the overall difference between two cumulative distribution functions: The absolute value of the area between them, for example. Or their integrated mean square difference. The Kolmogorov-Smirnov D is a particularly simple measure: It is defined as the *maximum value* of the absolute difference between two cumulative distribution

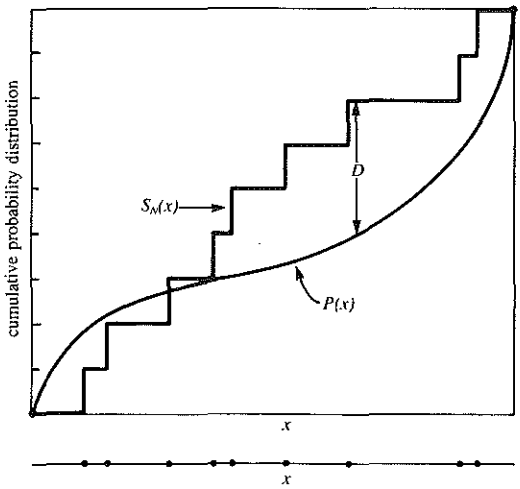


Figure 13.5.1. Kolmogorov-Smirnov statistic D. A measured distribution of values in x (shown as N dots on the lower abscissa) is to be compared with a theoretical distribution whose cumulative probability distribution is plotted as $P(x)$. A step-function cumulative probability distribution $S_N(x)$ is constructed, one which rises an equal amount at each measured point. D is the greatest distance between the two cumulative distributions.

functions. Thus, for comparing one data set's $S_N(x)$ to a known cumulative distribution function $P(x)$, the K–S statistic is

$$D = \max_{-\infty<x<\infty} |S_N(x) - P(x)| \tag{13.5.3}$$

while for comparing two different cumulative distribution functions $S_{N_1}(x)$ and $S_{N_2}(x)$, the K–S statistic is

$$D = \max_{-\infty<x<\infty} |S_{N_1}(x) - S_{N_2}(x)| \tag{13.5.4}$$

What makes the K–S statistic useful is that *its* distribution in the case of the null hypothesis (data sets drawn from the same distribution) can be calculated, at least to useful approximation, thus giving the significance of any observed nonzero value of D.

The function which enters into the calculation of the significance can be written as the following sum,

$$Q_{KS}(\lambda) = 2\sum_{j=1}^{\infty}(-)^{j-1}\, e^{-2j^2\lambda^2} \tag{13.5.5}$$

which is a monotonic function with the limiting values

$$Q_{KS}(0) = 1 \qquad Q_{KS}(\infty) = 0 \tag{13.5.6}$$

In terms of this function, the significance level of an observed value of D (as a disproof of the null hypothesis that the distributions are the same) is given approximately by the formulas

$$\text{Probability } (D > \text{observed}) = Q_{KS}(\sqrt{N}\, D) \tag{13.5.7}$$

for the case (13.5.3) of one distribution, where N is the number of data points, and

$$\text{Probability } (D > \text{observed}) = Q_{KS}\left(\sqrt{\frac{N_1 N_2}{N_1 + N_2}}\, D\right) \tag{13.5.8}$$

for the case (13.5.4) of two distributions, where N_1 is the number of data points in the first distribution, N_2 the number in the second.

The nature of the approximation involved in (13.5.7) and (13.5.8) is that it becomes asymptotically accurate as the N's become large. In practice, $N = 20$ is large enough, especially if you are being conservative and requiring a strong significance level (0.01 or smaller).

So, we have the following routines for the cases of one and two distributions:

```
#include <math.h>

static float maxarg1,maxarg2;
#define MAX(a,b) (maxarg1=(a),maxarg2=(b),(maxarg1) > (maxarg2) ?\
    (maxarg1) : (maxarg2))

void ksone(data,n,func,d,prob)
float data[],*d,*prob;
float (*func)();    /* ANSI: float (*func)(float); */
int n;
```

Given an array `data[1..n]`, and given a user-supplied function of a single variable `func` which is a cumulative distribution function ranging from 0 (for smallest values of its argument) to 1 (for largest values of its argument), this routine returns the K–S statistic `d`, and the significance level `prob`. Small values of `prob` show that the cumulative distribution function of `data` is significantly different from `func`. The array `data` is modified by being sorted into ascending order.

```
{
    int j;
    float fo=0.0,fn,ff,en,dt;
    void sort();
    float probks();

    sort(n,data);                    If the data are already sorted into ascending order, then
    en=n;                                this call can be omitted.
    *d=0.0;
    for (j=1;j<=n;j++) {             Loop over the sorted data points.
        fn=j/en;                     Data's c.d.f. after this step.
```

```
        ff=(*func)(data[j]);            Compare to the user-supplied function.
        dt = MAX(fabs(fo-ff),fabs(fn-ff));          Maximum distance.
        if (dt > *d) *d=dt;
        fo=fn;
    }
    *prob=probks(sqrt(en)*(*d));        Compute significance.
}
```

```
#include <math.h>

void kstwo(data1,n1,data2,n2,d,prob)
float data1[],data2[],*d,*prob;
int n1,n2;
```

Given an array `data1[1..n1]` and an array `data2[1..n2]`, this routine returns the K–S statistic `d`, and the significance level `prob` for the null hypothesis that the data sets are drawn from the same distribution. Small values of `prob` show that the cumulative distribution function of `data1` is significantly different from that of `data2`. The arrays `data1` and `data2` are modified by being sorted into ascending order.

```
{
    int j1=1,j2=1;
    float en1,en2,fn1=0.0,fn2=0.0,dt,d1,d2;
    void sort();
    float probks();

    en1=n1;
    en2=n2;
    *d=0.0;
    sort(n1,data1);
    sort(n2,data2);
    while (j1 <= n1 && j2 <= n2) {          If we are not done...
        if ((d1=data1[j1]) <= (d2=data2[j2])) {          Next step is in data1.
            fn1=(j1++)/en1;
        }
        if (d2 <= d1) {                                  Next step is in data2.
            fn2=(j2++)/en2;
        }
        if ((dt=fabs(fn2-fn1)) > *d) *d=dt;
    }
    *prob=probks(sqrt(en1*en2/(en1+en2))*(*d));          Compute significance.
}
```

Both of the above routines use the following routine for calculating the function Q_{KS}:

```
#include <math.h>

#define EPS1 0.001
#define EPS2 1.0e-8

float probks(alam)
float alam;
{
    int j;
    float a2,fac=2.0,sum=0.0,term,termbf=0.0;

    a2 = -2.0*alam*alam;
    for (j=1;j<=100;j++) {
        term=fac*exp(a2*j*j);
```

```
        sum += term;
        if (fabs(term) <= EPS1*termbf || fabs(term) <= EPS2*sum) return sum;
        fac = -fac;                    Alternating signs in sum.
        termbf=fabs(term);
    }
    return 1.0;                        Get here only by failing to converge.
}
```

REFERENCES AND FURTHER READING:

von Mises, Richard. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), Chapters IX(C) and IX(E).

13.6 Contingency Table Analysis of Two Distributions

In this section, and the next two sections, we deal with *measures of association* for two distributions. The situation is this: Each data point has two or more different quantities associated with it, and we want to know whether knowledge of one quantity gives us any demonstrable advantage in predicting the value of another quantity. In many cases, one variable will be an "independent" or "control" variable, and another will be a "dependent" or "measured" variable. Then, we want to know if the latter variable *is* in fact dependent on or *associated* with the former variable. If it is, we want to have some quantitative measure of the strength of the association. One often hears this loosely stated as the question of whether two variables are *correlated* or *uncorrelated*, but we will reserve those terms for a particular kind of association (linear, or at least monotonic), as discussed in §13.7 and §13.8.

Notice that, as in previous sections, the different concepts of significance and strength appear: The association between two distributions may be very significant even if that association is weak – if the quantity of data is large enough.

It is useful to distinguish among some different kinds of variables, with different categories forming a loose hierarchy.

- A variable is called *nominal* if its values are the members of some unordered set. For example, "state of residence" is a nominal variable that (in the U.S.) takes on one of 50 values; in astrophysics, "type of galaxy" is a nominal variable with the three values "spiral," "elliptical," and "irregular."
- A variable is termed *ordinal* if its values are the members of a discrete, but ordered, set. Examples are: grade in school, planetary order from the Sun (Mercury = 1, Venus = 2, ...), number of offspring. There need not be any concept of "equal metric distance" between the values of an ordinal variable, only that they be intrinsically ordered.

	1. red	2. green	...	
1. male	# of red males N_{11}	# of green males N_{12}	...	# of males $N_{1\cdot}$
2. female	# of red females N_{21}	# of green females N_{22}	...	# of females $N_{2\cdot}$
⋮	⋮	⋮	...	⋮
	# of red $N_{\cdot 1}$	# of green $N_{\cdot 2}$	...	total # N

Figure 13.6.1. Example of a contingency table for two nominal variables, here sex and color. The row and column marginals (totals) are shown. The variables are "nominal," i.e. the order in which their values are listed is arbitrary and does not affect the result of the contingency table analysis. If the ordering of values has some intrinsic meaning, then the variables are "ordinal" or "continuous," and correlation techniques (§13.7 – §13.8) can be utilized.

- We will call a variable *continuous* if its values are real numbers, as are times, distances, temperatures, etc. (Social scientists sometimes distinguish between *interval* and *ratio* continuous variables, but we do not find that distinction very compelling.)

A continuous variable can always be made into an ordinal one by binning it into ranges. If we choose to ignore the ordering of the bins, then we can turn it into a nominal variable. Nominal variables constitute the lowest type of the hierarchy, and therefore the most general. For example, a set of *several* continuous or ordinal variables can be turned, if crudely, into a single nominal variable, by coarsely binning each variable and then taking each distinct combination of bin assignments as a single nominal value. When multidimensional data are sparse, this is often the only sensible way to proceed.

The remainder of this section will deal with measures of association between *nominal* variables. For any pair of nominal variables, the data can be displayed as a *contingency table*, a table whose rows are labeled by the values of one nominal variable, whose columns are labeled by the values of the other nominal variable, and whose entries are nonnegative integers giving the number of observed events for each combination of row and column (see Figure 13.6.1). The analysis of association between nominal variables is thus called *contingency table analysis* or *crosstabulation analysis*.

We will introduce two different approaches. The first approach, based on the chi-square statistic, does a good job of characterizing the significance of association, but is only so-so as a measure of the strength (principally because its numerical values have no very direct interpretations). The second approach, based on the information-theoretic concept of *entropy*, says nothing at all about the significance of association (use chi-square for that!), but is capable of very elegantly characterizing the strength of an association already known to be significant.

Measures of Association Based on Chi-Square

Some notation first: Let N_{ij} denote the number of events which occur with the first variable x taking on its ith value, and the second variable y taking on its jth value. Let N denote the total number of events, the sum of all the N_{ij}'s. Let $N_{i\cdot}$ denote the number of events for which the first variable x takes on its ith value regardless of the value of y; $N_{\cdot j}$ is the number of events with the jth value of y regardless of x. So we have

$$N_{i\cdot} = \sum_j N_{ij} \qquad N_{\cdot j} = \sum_i N_{ij}$$
$$N = \sum_i N_{i\cdot} = \sum_j N_{\cdot j} \tag{13.6.1}$$

$N_{\cdot j}$ and $N_{i\cdot}$ are sometimes called the *row and column totals* or *marginals*, but we will use these terms cautiously since we can never keep straight which are the rows and which are the columns!

The null hypothesis is that the two variables x and y have no association. In this case, the probability of a particular value of x given a particular value of y should be the same as the probability of that value of x regardless of y. Therefore, in the null hypothesis, the expected number for any N_{ij}, which we will denote n_{ij}, can be calculated from only the row and column totals,

$$\frac{n_{ij}}{N_{\cdot j}} = \frac{N_{i\cdot}}{N} \qquad \text{which implies} \qquad n_{ij} = \frac{N_{i\cdot} N_{\cdot j}}{N} \tag{13.6.2}$$

Notice that if a column or row total is zero, then the expected number for all the entries in that column or row is also zero; in that case, the never-occurring bin of x or y should simply be removed from the analysis.

The chi-square statistic is now given by equation (13.5.1) which, in the present case, is summed over all entries in the table,

$$\chi^2 = \sum_{i,j} \frac{(N_{ij} - n_{ij})^2}{n_{ij}} \tag{13.6.3}$$

The number of degrees of freedom is equal to the number of entries in the table (product of its row size and column size) minus the number of constraints that have arisen from our use of the data itself to determine the n_{ij}. Each row total and column total is a constraint, except that this overcounts by one, since the total of the column totals and the total of the row totals both equal N, the total number of data points. Therefore, if the table is of size I by J, the number of degrees of freedom is $IJ - I - J + 1$. Equation (13.6.3), along with the chi-square probability function (§6.2) now give the significance of an association between the variables x and y.

Suppose there is a significant association. How do we quantify its strength, so that (e.g.) we can compare the strength of one association to another? The idea here is to find some reparametrization of χ^2 which maps it into some convenient interval, like 0 to 1, where the result is not dependent on the quantity of data that we happen to sample, but rather depends only on the underlying population from which the data were drawn. There are several different ways of doing this. Two of the more common are called *Cramer's V* and the *contingency coefficient C*.

The formula for Cramer's V is

$$V = \sqrt{\frac{\chi^2}{N \min(I-1, J-1)}} \tag{13.6.4}$$

where I and J are again the numbers of rows and columns, N is the total number of events. Cramer's V has the pleasant property that it lies between zero and one inclusive, equals zero when there is no association, and equals one only when the association is perfect: All the events in any row lie in one unique column, and vice versa. (In chess parlance, no two rooks, placed on a nonzero table entry, can capture each other.)

In the case of $I = J = 2$, Cramer's V is also referred to as the *phi* statistic.

The contingency coefficient C is defined as

$$C = \sqrt{\frac{\chi^2}{\chi^2 + N}} \tag{13.6.5}$$

It also lies between zero and one, but (as is apparent from the formula) it can never achieve the upper limit. While it can be used to compare the strength of association of two tables with the same I and J, its upper limit depends on I and J. Therefore it can never be used to compare tables of different sizes.

The trouble with both Cramer's V and with the contingency coefficient C is that, when they take on values in between their extremes, there is no very direct interpretation of what that value means. For example, you are in Las Vegas, and a friend tells you that there is a small, but significant, association between the color of a croupier's eyes and the occurrence of red and black on his roulette wheel. Cramer's V is about 0.028, your friend tells you. You know what the usual odds against you are (due to the green zero and double zero on the wheel). Is this association sufficient for you to make money? Don't ask us!

```
#include <math.h>

#define TINY 1.0e-30

void cntab1(nn,ni,nj,chisq,df,prob,cramrv,ccc)
int **nn,ni,nj;
float *chisq,*df,*prob,*cramrv,*ccc;
```

Given a two-dimensional contingency table in the form of an integer array `nn[1..ni][1..nj]`, this routine returns the chi-square `chisq`, the number of degrees of freedom `df`, the significance

level **prob** (small values indicating a significant association), and two measures of association, Cramer's V (**cramrv**) and the contingency coefficient C (**ccc**).

```
{
    int nnj,nni,j,i,minij;
    float sum=0.0,expctd,*sumi,*sumj,temp,gammq(),*vector();
    void free_vector();

    sumi=vector(1,ni);
    sumj=vector(1,nj);
    nni=ni;                                  Number of rows
    nnj=nj;                                  and columns.
    for (i=1;i<=ni;i++) {                    Get the row totals.
        sumi[i]=0.0;
        for (j=1;j<=nj;j++) {
            sumi[i] += nn[i][j];
            sum += nn[i][j];
        }
        if (sumi[i] == 0.0) --nni;           Eliminate any zero rows by reducing the number.
    }
    for (j=1;j<=nj;j++) {                    Get the column totals.
        sumj[j]=0.0;
        for (i=1;i<=ni;i++) sumj[j] += nn[i][j];
        if (sumj[j] == 0.0) --nnj;           Eliminate any zero columns.
    }
    *df=nni*nnj-nni-nnj+1;                   Corrected number of degrees of freedom.
    *chisq=0.0;
    for (i=1;i<=ni;i++) {                    Do the chi-square sum.
        for (j=1;j<=nj;j++) {
            expctd=sumj[j]*sumi[i]/sum;
            temp=nn[i][j]-expctd;
            *chisq += temp*temp/(expctd+TINY);        Here TINY guarantees that any elim-
        }                                             inated row or column will not
    }                                                 contribute to the sum.
    *prob=gammq(0.5*(*df),0.5*(*chisq));     Chi-square probability function.
    minij = nni < nnj ? nni-1 : nnj-1;
    *cramrv=sqrt(*chisq/(sum*minij));
    *ccc=sqrt(*chisq/(*chisq+sum));
    free_vector(sumj,1,nj);
    free_vector(sumi,1,ni);
}
```

Measures of Association Based on Entropy

Consider the game of "twenty questions," where by repeated yes/no questions you try to eliminate all except one correct possibility for an unknown object. Better yet, consider a generalization of the game, where you are allowed to ask multiple choice questions as well as binary (yes/no) ones. The categories in your multiple choice questions are supposed to be mutually exclusive and exhaustive (as are "yes" and "no").

The value to you of an answer increases with the number of possibilities that it eliminates. More specifically, an answer that eliminates all except a fraction p of the remaining possibilities can be assigned a value $-\ln p$ (a positive number, since $p < 1$). The purpose of the logarithm is to make the value additive, since (e.g.) one question which eliminates all but 1/6 of the possibilities is considered as good as two questions that, in sequence, reduce the number by factors 1/2 and 1/3.

So that is the value of an answer; but what is the value of a question? If there are I possible answers to the question ($i = 1, \ldots, I$) and the fraction of possibilities consistent with the ith answer is p_i (with the sum of the p_i's equal to one), then the value of the question is the expectation value of the value of the answer, denoted H,

$$H = -\sum_{i=1}^{I} p_i \ln(p_i) \tag{13.6.6}$$

In evaluating (13.6.6), note that

$$\lim_{p \to 0} p \ln(p) = 0 \tag{13.6.7}$$

The value H lies between 0 and $\ln I$. It is zero only when one of the p_i's is one, all the others zero: in this case, the question is valueless, since its answer is preordained. H takes on its maximum value when all the p_i's are equal, in which case the question is sure to eliminate all but a fraction $1/I$ of the remaining possibilities.

The value H is conventionally termed the *entropy* of the distribution given by the p_i's, a terminology borrowed from statistical physics.

So far we have said nothing about the association of two variables; but suppose we are deciding what question to ask next in the game and have to choose between two candidates, or possibly want to ask both in one order or another. Suppose that one question, x, has I possible answers, labeled by i, and that the other question, y, as J possible answers, labeled by j. Then the possible outcomes of asking both questions form a contingency table whose entries N_{ij}, when normalized by dividing by the total number of remaining possibilities N, give all the information about the p's. In particular, we can make contact with the notation (13.6.1) by identifying

$$\begin{aligned} p_{ij} &= \frac{N_{ij}}{N} \\ p_{i\cdot} &= \frac{N_{i\cdot}}{N} \qquad \text{(outcomes of question } x \text{ alone)} \\ p_{\cdot j} &= \frac{N_{\cdot j}}{N} \qquad \text{(outcomes of question } y \text{ alone)} \end{aligned} \tag{13.6.8}$$

The entropies of the questions x and y are respectively

$$H(x) = -\sum_i p_{i\cdot} \ln p_{i\cdot} \qquad H(y) = -\sum_j p_{\cdot j} \ln p_{\cdot j} \tag{13.6.9}$$

The entropy of the two questions together is

$$H(x,y) = -\sum_{i,j} p_{ij} \ln p_{ij} \tag{13.6.10}$$

Now what is the entropy of the question y *given* x (that is, if x is asked first)? It is the expectation value over the answers to x of the entropy of the restricted y distribution that lies in a single column of the contingency table (corresponding to the x answer):

$$H(y|x) = -\sum_i p_{i\cdot} \sum_j \frac{p_{ij}}{p_{i\cdot}} \ln \frac{p_{ij}}{p_{i\cdot}} = -\sum_{i,j} p_{ij} \ln \frac{p_{ij}}{p_{i\cdot}} \tag{13.6.11}$$

Correspondingly, the entropy of x given y is

$$H(x|y) = -\sum_j p_{\cdot j} \sum_i \frac{p_{ij}}{p_{\cdot j}} \ln \frac{p_{ij}}{p_{\cdot j}} = -\sum_{i,j} p_{ij} \ln \frac{p_{ij}}{p_{\cdot j}} \tag{13.6.12}$$

We can readily prove that the entropy of y given x is never more than the entropy of y alone, i.e. that asking x first can only reduce the usefulness of asking y (in which case the two variables are *associated*!):

$$\begin{aligned} H(y|x) - H(y) &= -\sum_{i,j} p_{ij} \ln \frac{p_{ij}/p_{i\cdot}}{p_{\cdot j}} \\ &= \sum_{i,j} p_{ij} \ln \frac{p_{\cdot j} p_{i\cdot}}{p_{ij}} \\ &\le \sum_{i,j} p_{ij} \left(\frac{p_{\cdot j} p_{i\cdot}}{p_{ij}} - 1 \right) \\ &= \sum_{i,j} p_{i\cdot} p_{\cdot j} - \sum_{i,j} p_{ij} \\ &= 1 - 1 = 0 \end{aligned} \tag{13.6.13}$$

where the inequality follows from the fact

$$\ln w \le w - 1 \tag{13.6.14}$$

We now have everything we need to define a measure of the "dependency" of y on x, that is to say a measure of association. This measure is sometimes

called the *uncertainty coefficient* of y. We will denote it as $U(y|x)$,

$$U(y|x) \equiv \frac{H(y) - H(y|x)}{H(y)} \tag{13.6.15}$$

This measure lies between zero and one, with the value 0 indicating that x and y have no association, the value 1 indicating that knowledge of x completely predicts y. For in-between values, $U(y|x)$ gives the fraction of y's entropy $H(y)$ that is lost if x is already known (i.e., that is redundant with the information in x). In our game of "twenty questions," $U(y|x)$ is the fractional loss in the utility of question y if question x is to be asked first.

If we wish to view x as the dependent variable, y as the independent one, then interchanging x and y we can of course define the dependency of x on y,

$$U(x|y) \equiv \frac{H(x) - H(x|y)}{H(x)} \tag{13.6.16}$$

If we want to treat x and y symmetrically, then the useful combination turns out to be

$$U(x,y) \equiv 2\left[\frac{H(y) + H(x) - H(x,y)}{H(x) + H(y)}\right] \tag{13.6.17}$$

If the two variables are completely independent, then $H(x,y) = H(x) + H(y)$, so (13.6.17) vanishes. If the two variables are completely dependent, then $H(x) = H(y) = H(x,y)$, so (13.6.16) equals unity. In fact, you can use the identities (easily proved from equations 13.6.9–13.6.12)

$$H(x,y) = H(x) + H(y|x) = H(y) + H(x|y) \tag{13.6.18}$$

to show that

$$U(x,y) = \frac{H(x)U(x|y) + H(y)U(y|x)}{H(x) + H(y)} \tag{13.6.19}$$

i.e. that the symmetrical measure is just a weighted average of the two asymmetrical measures (13.6.15) and (13.6.16), weighted by the entropy of each variable separately.

Here is a program for computing all the quantities discussed, $H(x)$, $H(y)$, $H(x|y)$, $H(y|x)$, $H(x,y)$, $U(x|y)$, $U(y|x)$, and $U(x,y)$:

```
#include <math.h>

#define TINY 1.0e-30

void cntab2(nn,ni,nj,h,hx,hy,hygx,hxgy,uygx,uxgy,uxy)
int ni,nj,**nn;
float *h,*hx,*hy,*hygx,*hxgy,*uygx,*uxgy,*uxy;
```

Given a two-dimensional contingency table in the form of an integer array **nn[i][j]**, where **i** labels the x variable and ranges from **1** to **ni**, **j** labels the y variable and ranges from **1** to **nj**, this routine returns the entropy **h** of the whole table, the entropy **hx** of the x distribution, the entropy **hy** of the y distribution, the entropy **hygx** of y given x, the entropy **hxgy** of x given y, the dependency **uygx** of y on x (eq. 13.6.15), the dependency **uxgy** of x on y (eq. 13.6.16), and the symmetrical dependency **uxy** (eq. 13.6.17).

```
{
    int i,j;
    float sum=0.0,p,*sumi,*sumj,*vector();
    void free_vector();

    sumi=vector(1,ni);
    sumj=vector(1,nj);
    for (i=1;i<=ni;i++) {                          Get the row totals.
        sumi[i]=0.0;
        for (j=1;j<=nj;j++) {
            sumi[i] += nn[i][j];
            sum += nn[i][j];
        }
    }
    for (j=1;j<=nj;j++) {                          Get the column totals.
        sumj[j]=0.0;
        for (i=1;i<=ni;i++)
            sumj[j] += nn[i][j];
    }
    *hx=0.0;                                       Entropy of the x distribution,
    for (i=1;i<=ni;i++)
        if (sumi[i]) {
            p=sumi[i]/sum;
            *hx -= p*log(p);
        }
    *hy=0.0;                                       and of the y distribution.
    for (j=1;j<=nj;j++)
        if (sumj[j]) {
            p=sumj[j]/sum;
            *hy -= p*log(p);
        }
    *h=0.0;
    for (i=1;i<=ni;i++)                            Total entropy: loop over both x
        for (j=1;j<=nj;j++)                        and y.
            if (nn[i][j]) {
                p=nn[i][j]/sum;
                *h -= p*log(p);
            }
    *hygx=(*h)-(*hx);                              Uses equation (13.6.18),
    *hxgy=(*h)-(*hy);                              as does this.
    *uygx=(*hy-*hygx)/(*hy+TINY);                  Equation (13.6.15).
    *uxgy=(*hx-*hxgy)/(*hx+TINY);                  Equation (13.6.16).
    *uxy=2.0*(*hx+*hy-*h)/(*hx+*hy+TINY);          Equation (13.6.17).
    free_vector(sumj,1,nj);
    free_vector(sumi,1,ni);
}
```

REFERENCES AND FURTHER READING:

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill), §16.1.

Downie, N.M., and Heath, R.W. 1965, *Basic Statistical Methods*, 2nd ed. (New York: Harper and Row), pp. 196, 210.

Fano, Robert M. 1961, *Transmission of Information* (New York: Wiley and MIT Press), Chapter 2.

13.7 Linear Correlation

We next turn to measures of association between variables that are ordinal or continuous, rather than nominal. Most widely used is the *linear correlation coefficient*. For pairs of quantities (x_i, y_i), $i = 1, \ldots, N$, the linear correlation coefficient r (also called *Pearson's* r) is given by the formula

$$r = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_i (x_i - \overline{x})^2} \sqrt{\sum_i (y_i - \overline{y})^2}} \tag{13.7.1}$$

where, as usual, $\overline{x}$ is the mean of the x_i's, $\overline{y}$ is the mean of the y_i's.

The value of r lies between -1 and 1, inclusive. It takes on a value of 1, termed "complete positive correlation," when the data points lie on a perfect straight line with positive slope, with x and y increasing together. The value 1 holds independent of the magnitude of the slope. If the data points lie on a perfect straight line with negative slope, y decreasing as x increases, then r has the value -1; this is called "complete negative correlation." A value of r near zero indicates that the variables x and y are *uncorrelated*.

When a correlation is known to be significant, r is one conventional way of summarizing its strength. In fact, the value of r can be translated into a statement about what residuals (root mean square deviations) are to be expected if the data are fitted to a straight line by the least-squares method (see §14.2, especially equations 14.2.11 and 14.2.12). Unfortunately, r is a rather poor statistic for deciding *whether* an observed correlation is statistically significant, and/or whether one observed correlation is significantly stronger than another. The reason is that r is ignorant of the individual distributions of x and y, so there is no universal way to compute its distribution in the case of the null hypothesis.

About the only general statement that can be made is this: If the null hypothesis is that x and y are uncorrelated, and if the distributions for x and y each have enough convergent moments ("tails" die off sufficiently rapidly), and if N is large (typically > 20), then r is distributed approximately normally, with a mean of zero and a standard deviation of $1/\sqrt{N}$. In that case, the

(double-sided) significance of the correlation, that is, the probability that $|r|$ should be larger than its observed value in the null hypothesis, is

$$\text{erfc}\left(\frac{|r|\sqrt{N}}{\sqrt{2}}\right) \tag{13.7.2}$$

where $\text{erfc}(x)$ is the complementary error function, equation (6.2.8), computed by the routines `erfc` or `erfcc` of §6.2. A small value of (13.7.2) indicates that the two distributions are significantly correlated.

Most statistics books try to go beyond (13.7.2) and give additional statistical tests which can be made using r. In almost all cases, however, these tests are valid only for a very special class of hypotheses, namely that the distributions of x and y jointly form a *binormal* or *two-dimensional Gaussian* distribution around their mean values, with joint probability density

$$p(x,y)\,dxdy = \text{const.} \times \exp\left[-\frac{1}{2}(a_{11}x^2 - 2a_{12}xy + a_{22}y^2)\right]\,dxdy \tag{13.7.3}$$

where a_{11}, a_{12}, and a_{22} are arbitrary constants. For this distribution r has the value

$$r = -\frac{a_{12}}{\sqrt{a_{11}a_{22}}} \tag{13.7.4}$$

There are occasions when (13.7.3) may be known to be a good model of the data. There may be other occasions when we are willing to take (13.7.3) as at least a rough and ready guess, since many two-dimensional distributions do resemble a binormal distribution, at least not too far out on their tails. In either situation, we can use (13.7.3) to go beyond (13.7.2) in any of several directions:

First, we can allow for the possibility that the number N of data points is not large. Here, it turns out that the statistic

$$t = r\sqrt{\frac{N-2}{1-r^2}} \tag{13.7.5}$$

is distributed in the null case (of no correlation) like Student's t-distribution with $\nu = N-2$ degrees of freedom, whose two-sided significance level is given by $1 - A(t|\nu)$ (equation 6.3.7). As N becomes large, this significance and (13.7.2) become asymptotically the same, so that one never does worse by using (13.7.5), even if the binormal assumption is not well substantiated.

Second, when N is moderately large (≥ 10), we can compare whether the difference of two significantly nonzero r's, e.g., from different experiments, is itself significant. In other words, we can quantify whether a change in some

control variable significantly alters an existing correlation between two other variables. This is done by using *Fisher's z-transformation* to associate each measured r with a corresponding z,

$$z = \frac{1}{2}\ln\left(\frac{1+r}{1-r}\right) \tag{13.7.6}$$

Then, each z is approximately normally distributed with a mean value

$$\overline{z} = \frac{1}{2}\left[\ln\left(\frac{1+r_{true}}{1-r_{true}}\right) + \frac{r_{true}}{N-1}\right] \tag{13.7.7}$$

where r_{true} is the actual or population value of the correlation coefficient, and with a standard deviation

$$\sigma(z) \approx \frac{1}{\sqrt{N-3}} \tag{13.7.8}$$

Equations (13.7.7) and (13.7.8), when they are valid, give several useful statistical tests. For example, the significance level at which a measured value of r differs from some hypothesized value r_{true} is given by

$$\operatorname{erfc}\left(\frac{|z-\overline{z}|\sqrt{N-3}}{\sqrt{2}}\right) \tag{13.7.9}$$

where z and $\overline{z}$ are given by (13.7.6) and (13.7.7), with small values of (13.7.9) indicating a significant difference. Similarly, the significance of a difference between two measured correlation coefficients r_1 and r_2 is

$$\operatorname{erfc}\left(\frac{|z_1 - z_2|}{\sqrt{2}\sqrt{\frac{1}{N_1-3}+\frac{1}{N_2-3}}}\right) \tag{13.7.10}$$

where z_1 and z_2 are obtained from r_1 and r_2 using (13.7.6), and where N_1 and N_2 are respectively the number of data points in the measurement of r_1 and r_2.

All of the significances above are two-sided. If you wish to disprove the null hypothesis in favor of a one-sided hypothesis, such as that $r_1 > r_2$ (where the sense of the inequality was decided *a priori*), then (i) if your measured r_1 and r_2 have the *wrong* sense, you have failed to demonstrate your one-sided hypothesis, but (ii) if they have the right ordering, you can multiply the significances given above by 0.5, which makes them more significant.

But keep in mind: These interpretations of the r statistic can be completely meaningless if the joint probability distribution of your variables x and y is too different from a binormal distribution.

```
#include <math.h>

#define TINY 1.0e-20                    Will regularize the unusual case of complete correlation.

void pearsn(x,y,n,r,prob,z)
float x[],y[],*r,*prob,*z;
int n;
Given two arrays x[1..n] and y[1..n], this routine computes their correlation coefficient
r (returned as r), the significance level at which the null hypothesis of zero correlation is
disproved (prob whose small value indicates a significant correlation), and Fisher's z (returned
as z), whose value can be used in further statistical tests as described above.
{
    int j;
    float yt,xt,t,df;
    float syy=0.0,sxy=0.0,sxx=0.0,ay=0.0,ax=0.0;
    float betai(),erfcc();

    for (j=1;j<=n;j++) {                Find the means.
        ax += x[j];
        ay += y[j];
    }
    ax /= n;
    ay /= n;
    for (j=1;j<=n;j++) {                Compute the correlation coefficient.
        xt=x[j]-ax;
        yt=y[j]-ay;
        sxx += xt*xt;
        syy += yt*yt;
        sxy += xt*yt;
    }
    *r=sxy/sqrt(sxx*syy);
    *z=0.5*log((1.0+(*r)+TINY)/(1.0-(*r)+TINY));          Fisher's z transformation.
    df=n-2;
    t=(*r)*sqrt(df/((1.0-(*r)+TINY)*(1.0+(*r)+TINY)));    Equation (13.7.5).
    *prob=betai(0.5*df,0.5,df/(df+t*t));                  Student's t probability.
/*    *prob=erfcc(fabs((*z)*sqrt(n-1.0))/1.4142136)    */
For large n, this easier computation of prob, using the short routine erfcc, would give approximately the
same value.
}
```

REFERENCES AND FURTHER READING:

Downie, N.M., and Heath, R.W. 1965, *Basic Statistical Methods*, 2nd ed. (New York: Harper and Row), Chapters 7 and 13.

Hoel, P.G. 1971, *Introduction to Mathematical Statistics*, 4th ed. (New York: Wiley), Chapter 7.

von Mises, Richard. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), Chapters IX(A) and IX(B).

Korn, G.A., and Korn, T.M. 1968, *Mathematical Handbook for Scientists and Engineers*, 2nd ed. (New York: McGraw-Hill), §19.7.

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill), §18.2.

13.8 Nonparametric or Rank Correlation

It is precisely the uncertainty in interpreting the significance of the linear correlation coefficient r that leads us to the important concepts of *nonparametric* or *rank correlation*. As before, we are given N pairs of measurements (x_i, y_i). Before, difficulties arose from the fact that we did not necessarily know the probability distribution function from which the x_i's or y_i's were drawn.

The key concept of nonparametric correlation is this: If we replace the value of each x_i by the value of its *rank* among all the other x_i's in the sample, that is, $1, 2, 3, \ldots, N$, then the resulting list of numbers will be drawn from a perfectly known distribution function, namely uniformly from the integers between 1 and N, inclusive. Better than uniformly, in fact, since if the x_i's are all distinct, then each integer will occur precisely once. If some of the x_i's have identical values, it is conventional to assign to all these "ties" the mean of the ranks that they would have had if their values had been slightly different. This *midrank* will sometimes be an integer, sometimes a half-integer. In all cases the sum of all assigned ranks will be the same as the sum of the integers from 1 to N, namely $\frac{1}{2}N(N+1)$.

Of course we do exactly the same procedure for the y_i's, replacing each value by its rank among the other y_i's in the sample.

Now we are free to invent statistics for detecting correlation between uniform sets of integers between 1 and N, keeping in mind the possibility of ties in the ranks. There is, of course, some loss of information in replacing the original numbers by ranks. We could construct some rather artificial examples where a correlation could be detected parametrically (e.g. in the linear correlation coefficient r), but cannot be detected nonparametrically. Such examples are very rare in real life, however, and the slight loss of information in ranking is a small price to pay for a very major advantage: When a correlation is demonstrated to be present nonparametrically, then it is really there! (That is, to a certainty level which depends on the significance chosen.) Nonparametric correlation is more robust than linear correlation, more resistant to unplanned defects in the data, in the same sort of sense that the median is more robust than the mean. For more on the concept of robustness, see §14.6.

As always in statistics, some particular choices of a statistic have already been invented for us and consecrated, if not beatified, by popular use. We will discuss two, the *Spearman rank-order correlation coefficient* (r_s), and *Kendall's tau* (τ).

Spearman Rank-Order Correlation Coefficient

Let R_i be the rank of x_i among the other x's, S_i be the rank of y_i among the other y's, ties being assigned the appropriate midrank as described above. Then the rank-order correlation coefficient is defined to be the linear

correlation coefficient of the ranks, namely,

$$r_s = \frac{\sum_i (R_i - \overline{R})(S_i - \overline{S})}{\sqrt{\sum_i (R_i - \overline{R})^2} \sqrt{\sum_i (S_i - \overline{S})^2}} \tag{13.8.1}$$

The significance of a nonzero value of r_s is tested by computing

$$t = r_s \sqrt{\frac{N-2}{1-r_s^2}} \tag{13.8.2}$$

which is distributed approximately as Student's distribution with $N-2$ degrees of freedom. A key point is that this approximation does not depend on the original distribution of the x's and y's; it is always the same approximation, and always pretty good.

It turns out that r_s is closely related to another conventional measure of nonparametric correlation, the so-called *sum squared difference of ranks*, defined as

$$D = \sum_{i=1}^{N} (R_i - S_i)^2 \tag{13.8.3}$$

(This D is sometimes denoted D^{**}, where the asterisks are used to indicate that ties are treated by midranking.)

When there are no ties in the data, then the exact relation between D and r_s is

$$r_s = 1 - \frac{6D}{N^3 - N} \tag{13.8.4}$$

When there are ties, then the exact relation is slightly more complicated: Let f_k be the number of ties in the kth group of ties among the R_i's, and let g_m be the number of ties in the mth group of ties among the S_i's. Then it turns out that

$$r_s = \frac{1 - \frac{6}{N^3 - N}\left[D + \frac{1}{12}\sum_k (f_k^3 - f_k) + \frac{1}{12}\sum_m (g_m^3 - g_m)\right]}{\left[1 - \frac{\sum_k (f_k^3 - f_k)}{N^3 - N}\right]^{1/2} \left[1 - \frac{\sum_m (g_m^3 - g_m)}{N^3 - N}\right]^{1/2}} \tag{13.8.5}$$

holds exactly. Notice that if all the f_k's and all the g_m's are equal to one, meaning that there are no ties, then equation (13.8.5) reduces to equation (13.8.4).

In (13.8.2) we gave a t-statistic which tests the significance of a nonzero r_s. It is also possible to test the significance of D directly. The expectation value of D in the null hypothesis of uncorrelated data sets is

$$\overline{D} = \frac{1}{6}(N^3 - N) - \frac{1}{12}\sum_k (f_k^3 - f_k) - \frac{1}{12}\sum_m (g_m^3 - g_m) \tag{13.8.6}$$

its variance is

$$\begin{aligned} \mathrm{Var}(D) = & \frac{(N-1)N^2(N+1)^2}{36} \\ & \times \left[1 - \frac{\sum_k (f_k^3 - f_k)}{N^3 - N}\right]\left[1 - \frac{\sum_m (g_m^3 - g_m)}{N^3 - N}\right] \end{aligned} \tag{13.8.7}$$

and it is approximately normally distributed, so that the significance level is a complementary error function (cf. equation 13.7.2). In the program that follows, we calculate both the significance level obtained by using (13.8.2) and the significance level obtained by using (13.8.7); their discrepancy will give you an idea of how good the approximations are. You will also notice that we break off the task of assigning ranks (including tied midranks) into a separate function, `crank`.

```
#include <math.h>

static float sqrarg;
#define SQR(a) (sqrarg=(a),sqrarg*sqrarg)

void spear(data1,data2,n,d,zd,probd,rs,probrs)
float data1[],data2[];
float *d,*zd,*probd,*rs,*probrs;
int n;
```

Given two data arrays, `data1[1..n]` and `data2[1..n]`, this routine returns their sum-squared difference of ranks as `d`, the number of standard deviations by which `d` deviates from its null-hypothesis expected value as `zd`, the two-sided significance level of this deviation as `probd`, Spearman's rank correlation r_s as `rs`, and the two-sided significance level of its deviation from zero as `probrs`. The external routines `crank` (below) and `sort2` (§8.2) are used. A small value of either `probd` or `probrs` indicates a significant correlation (`rs` positive) or anticorrelation (`rs` negative).

```
{
    int j;
    float vard,t,sg,sf,fac,en3n,en,df,aved,*wksp1,*wksp2;
    void sort2(),crank(),free_vector();
    float erfcc(),betai(),*vector();

    wksp1=vector(1,n);
    wksp2=vector(1,n);
    for (j=1;j<=n;j++) {
        wksp1[j]=data1[j];
        wksp2[j]=data2[j];
    }
    sort2(n,wksp1,wksp2);
    crank(n,wksp1,&sf);
    sort2(n,wksp2,wksp1);
    crank(n,wksp2,&sg);
```

Sort each of the data arrays, and convert the entries to ranks. The values `sf` and `sg` return the sums $\sum(f_k^3 - f_k)$ and $\sum(g_m^3 - g_m)$ respectively.

```
    *d=0.0;
    for (j=1;j<=n;j++)                          Sum the squared difference of ranks.
        *d += SQR(wksp1[j]-wksp2[j]);
    en=n;
    en3n=en*en*en-en;
    aved=en3n/6.0-(sf+sg)/12.0;                 Expectation value of D,
    fac=(1.0-sf/en3n)*(1.0-sg/en3n);
    vard=((en-1.0)*en*en*SQR(en+1.0)/36.0)*fac; and variance of D give
    *zd=(*d-aved)/sqrt(vard);                   number of standard deviations,
    *probd=erfcc(fabs(*zd)/1.4142136);          and significance.
    *rs=(1.0-(6.0/en3n)*(*d+(sf+sg)/12.0))/sqrt(fac); Rank correlation coefficient,
    fac=(*rs+1.0)*(1.0-(*rs));
    if (fac > 0.0) {
        t=(*rs)*sqrt((en-2.0)/fac);             and its t value,
        df=en-2.0;
        *probrs=betai(0.5*df,0.5,df/(df+t*t));  give its significance.
    } else
        *probrs=0.0;
    free_vector(wksp2,1,n);
    free_vector(wksp1,1,n);
}


void crank(n,w,s)
float w[],*s;
int n;
Given a sorted array w[1..n], replaces the elements by their rank, including midranking of
ties, and returns as s the sum of f^3 - f, where f is the number of elements in each tie.
{
    int j=1,ji,jt;
    float t,rank;

    *s=0.0;
    while (j < n) {
        if (w[j+1] != w[j]) {                   Not a tie.
            w[j]=j;
            ++j;
        } else {                                A tie:
            for (jt=j+1;jt<=n && w[jt]==w[j];jt++);  How far does it go?
            rank=0.5*(j+jt-1);                  This is the mean rank of the tie,
            for (ji=j;ji<=(jt-1);ji++) w[ji]=rank;   so enter it into all the tied entries,
            t=jt-j;
            *s += t*t*t-t;                      and update s.
            j=jt;
        }
    }
    if (j == n) w[n]=n;                         If the last element was not tied, this is its rank.
}
```

Kendall's Tau

Kendall's τ is even more nonparametric than Spearman's r_s or D. Instead of using the numerical difference of ranks, it only uses the relative ordering of ranks: higher in rank, lower in rank, or the same in rank. But in that case we don't even have to rank the data! Ranks will be higher, lower, or the same if and only if the values are larger, smaller, or equal, respectively. Since it uses a "weaker" property of the data there can be applications in which τ is more robust than r_s; however, since it throws away information that is available to r_s, there can also be applications where τ is less powerful than

r_s (i.e., τ fails to find an actual correlation that r_s does find). On balance, we prefer r_s as being the more straightforward nonparametric test, but both statistics are in general use.

To define τ, we start with the N data points (x_i, y_i). Now consider all $\frac{1}{2}N(N-1)$ *pairs* of data points, where a data point cannot be paired with itself, and where the points in either order count as one pair. We call a pair *concordant* if the relative ordering of the ranks of the two x's (or for that matter the two x's themselves) is the same as the relative ordering of the ranks of the two y's (or for that matter the two y's themselves). We call a pair *discordant* if the relative ordering of the ranks of the two x's is opposite from the relative ordering of the ranks of the two y's. If there is a tie in either the ranks of the two x's or the ranks of the two y's, then we don't call the pair either concordant or discordant. If the tie is in the x's, we will call the pair an "extra y" pair. If the tie is in the y's, we will call the pair an "extra x pair." If the tie is in both the x's and the y's, we don't call the pair anything at all. Are you still with us?

Kendall's τ is now the following simple combination of these various counts:

$$\tau = \frac{\text{concordant} - \text{discordant}}{\sqrt{\text{concordant} + \text{discordant} + \text{extra-}y}\ \sqrt{\text{concordant} + \text{discordant} + \text{extra-}x}} \tag{13.8.8}$$

You can easily convince yourself that this must lie between 1 and -1, and that it takes on the extreme values only for complete rank agreement or complete rank reversal, respectively.

More important, Kendall has worked out, from the combinatorics, the approximate distribution of τ in the null hypothesis of no association between x and y. In this case τ is approximately normally distributed, with zero expectation value and a variance of

$$\text{Var}(\tau) = \frac{4N + 10}{9N(N-1)} \tag{13.8.9}$$

The following program proceeds according to the above description, and therefore loops over all pairs of data points. Beware: This is an $O(N^2)$ algorithm, unlike the algorithm for r_s, whose dominant sort operations are of order $N \log N$. If you are routinely computing Kendall's τ for data sets of more than a few thousand points, you may be in for some serious computing. If, however, you are willing to bin your data into a moderate number of bins, then keep reading.

```
#include <math.h>

void kendl1(data1,data2,n,tau,z,prob)
float data1[],data2[],*tau,*z,*prob;
int n;
```

Given data arrays **data1[1..n]** and **data2[1..n]**, this program returns Kendall's τ as **tau**, its number of standard deviations from zero as **z**, and its two-sided significance level as **prob**. Small values of **prob** indicate a significant correlation (**tau** positive) or anticorrelation (**tau** negative).

```
{
    int n2=0,n1=0,k,j,is=0;
    float svar,aa,a2,a1;
    float erfcc();

    for (j=1;j<n;j++) {                    Loop over first member of pair,
        for (k=(j+1);k<=n;k++) {           and second member.
            a1=data1[j]-data1[k];
            a2=data2[j]-data2[k];
            aa=a1*a2;
            if (aa) {                      Neither array has a tie.
                ++n1;
                ++n2;
                aa > 0.0 ? ++is : --is;
            } else {                       One or both arrays have ties.
                if (a1) ++n1;              An "extra x" event.
                if (a2) ++n2;              An "extra y" event.
            }
        }
    }
    *tau=is/(sqrt((double) n1)*sqrt((double) n2));          Equation (13.8.8).
    svar=(4.0*n+10.0)/(9.0*n*(n-1.0));     Equation (13.8.9).
    *z=(*tau)/sqrt(svar);
    *prob=erfcc(fabs(*z)/1.4142136);       Significance.
}
```

Sometimes it happens that there are only a few possible values each for x and y. In that case, the data can be recorded as a contingency table (see §13.6) which gives the number of data points for each contingency of x and y.

Spearman's rank-order correlation coefficient is not a very natural statistic under these circumstances, since it assigns to each x and y bin a not-very-meaningful midrank value and then totals up vast numbers of identical rank differences. Kendall's tau, on the other hand, with its simple counting, remains quite natural. Furthermore, its $O(N^2)$ algorithm is no longer a problem, since we can arrange for it to loop over pairs of contingency table entries (each containing many data points) instead of over pairs of data points. This is implemented in the program that follows.

Note that Kendall's tau can only be applied to contingency tables where both variables are *ordinal*, i.e. well-ordered, and that it looks specifically for monotonic correlations, not for arbitrary associations. These two properties make it less general than the methods of §13.6, which applied to *nominal*, i.e. unordered, variables and arbitrary associations.

Comparing **kendl1** above with **kendl2** below, you will see that we have "floated" a number of variables. This is because the number of events in a contingency table might be sufficiently large as to cause overflows in some of

the integer arithmetic, while the number of individual data points in a list could not possibly be that large [for an $O(N^2)$ routine!].

```
#include <math.h>

void kendl2(tab,i,j,tau,z,prob)
float **tab,*tau,*z,*prob;
int i,j;
```

Given a two-dimensional table `tab[1..i][1..j]`, such that `tab[k][l]` contains the number of events falling in bin `k` of one variable and bin `l` of another, this program returns Kendall's τ as `tau`, its number of standard deviations from zero as `z`, and its two-sided significance level as `prob`. Small values of `prob` indicate a significant correlation (`tau` positive) or anticorrelation (`tau` negative) between the two variables. Although `tab` is a real array, it will normally contain integral values.

```
{
    int nn,mm,m2,m1,lj,li,l,kj,ki,k;
    float svar,s=0.0,points,pairs,en2=0.0,en1=0.0;
    float erfcc();

    nn=i*j;                                  Total number of entries in contingency table.
    points=tab[i][j];
    for (k=0;k<=nn-2;k++) {                  Loop over entries in table,
        ki=(k/j);                            decoding a row
        kj=k-j*ki;                           and a column.
        points += tab[ki+1][kj+1];           Increment the total count of events.
        for (l=k+1;l<=nn-1;l++) {            Loop over other member of the pair,
            li=l/j;                          decoding its row
            lj=l-j*li;                       and column.
            mm=(m1=li-ki)*(m2=lj-kj);
            pairs=tab[ki+1][kj+1]*tab[li+1][lj+1];
            if (mm) {                        Not a tie.
                en1 += pairs;
                en2 += pairs;
                s += (mm > 0 ? pairs : -pairs);        Concordant or discordant.
            } else {
                if (m1) en1 += pairs;
                if (m2) en2 += pairs;
            }
        }
    }
    *tau=s/sqrt(en1*en2);
    svar=(4.0*points+10.0)/(9.0*points*(points-1.0));
    *z=(*tau)/sqrt(svar);
    *prob=erfcc(fabs(*z)/1.4142136);
}
```

REFERENCES AND FURTHER READING:

Lehmann, E.L. 1975, *Nonparametrics: Statistical Methods Based on Ranks* (San Francisco: Holden-Day).

Downie, N.M., and Heath, R.W. 1965, *Basic Statistical Methods*, 2nd ed. (New York: Harper and Row), pp. 206–209.

SPSS: Statistical Package for the Social Sciences, 2nd ed., by Norman H. Nie, et al. (New York: McGraw-Hill), §18.3.

13.9 Smoothing of Data

The concept of "smoothing" data lies in a murky area, just beyond the fringe of these better posed and more highly recommended techniques:

- Least squares fitting to a parametric model, which is covered in some detail in Chapter 14.
- Optimal filtering of a noisy signal, see §12.6.
- Letting it all hang out. (Have you considered showing the data as they actually are?)

On the other hand, it is useful to have some techniques available that are more objective than

- Draftsman's license. "The smooth curve was drawn by eye through the original data." (Through each individual data point? Or through the forest of scattered points? By a draftsman? Or by someone who knows what hypothesis the data are supposed to substantiate?)

Data smoothing is probably most justified when it is used simply as a graphical technique, to guide the eye through a forest of data points all with large error bars. In this case, the individual points and their error bars should be plotted on the same graph, and no quantitative claims should be made on the basis of the smoothed curve. Data smoothing is least justified when it is used subjectively to massage the data this way and that, until some feature in the smoothed curve emerges and is pounced on in support of an hypothesis.

Data smoothing is what we would call "semi-parametric." It clearly involves some notion of "averaging" the measured dependent (y) variable, which is parametric. Smoothing a set of values will not, in general, be the same as smoothing their logarithms. You have to think about which is closer to your needs. On the other hand, smoothing is not supposed to be tied to any particular functional form $y(x)$, or to any particular parametrization of the x axis. Smoothing is art, not science.

Here is a program for smoothing an array of ordinates (y's) that are in order of increasing abscissas (x's), but without using the abscissas themselves. The program pretends that the abscissas are equally spaced, as they are if reparametrized to the variable "point number." It removes any linear trend, and then uses a Fast Fourier Transform to low-pass filter the data. The linear trend is reinserted at the end. One user-specified constant enters: the "amount of smoothing," specified as the number of points over which the data should be smoothed (not necessarily an integer). Zero gives no smoothing at all, while any value larger than about half the number of data points will render the data virtually featureless. The program gives results that are generally in accord with the notion "draw a smooth curve through these scattered points," and is at least arguably objective in doing so. A sample of its output is shown in Figure 13.9.1.

```
#include <math.h>

void smooft(y,n,pts)
float y[],pts;
int n;
```

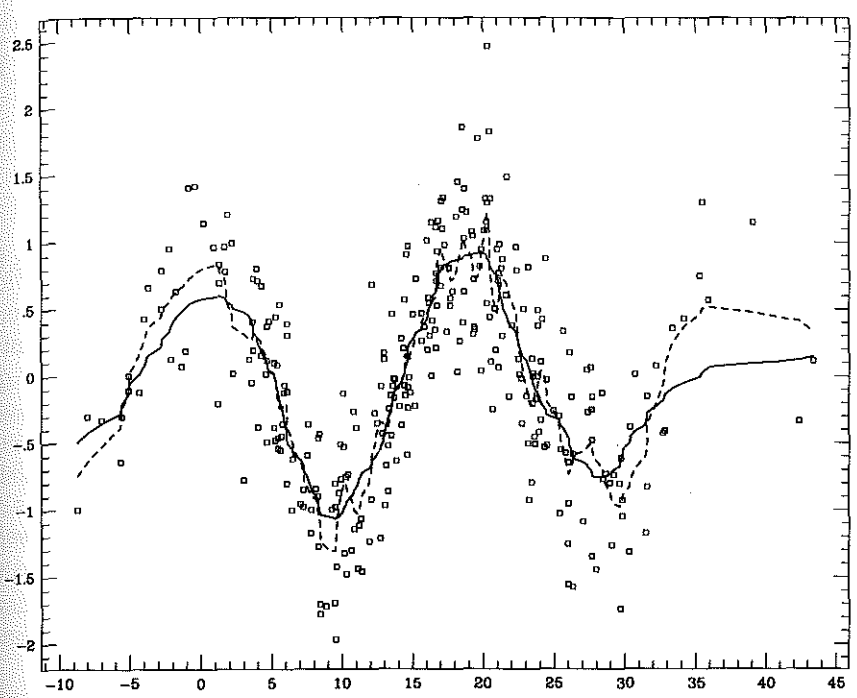


Figure 13.9.1. Smoothing of data with the routine **smooft**. The open squares are noisy data points, nonuniformly sampled with the greatest sampling density towards the middle of the abscissa. The dotted curve is obtained with smoothing parameter **pts**= 10.0 (averaging approximately 10 points); the solid curve has **pts**= 30.0.

Smooths an array y of n data points, with a window whose full width is of order **pts** neighboring points, a user supplied value. y is modified. y should be dimensioned at least of length [1..mp], where mp $\geq$ integral power of two $\geq$ n+2*pts.

```
{
    int nmin,m=2,mo2,k,j;
    float yn,y1,rn1,fac,cnst;
    void realft();

    nmin=n+(int) (2.0*pts+0.5);          Minimum size including buffer against wrap around.
    while (m < nmin) m *= 2;             Find the next larger power of 2.
    cnst=pts/m,cnst=cnst*cnst;           Useful constant below.
    y1=y[1];
    yn=y[n];
    rn1=1.0/(n-1);
    for (j=1;j<=n;j++)                   Remove the linear trend and transfer data.
        y[j] += (-rn1*(y1*(n-j)+yn*(j-1)));
    for (j=n+1;j<=m;j++) y[j]=0.0;       Zero pad.
    mo2=m >> 1;
    realft(y,mo2,1);                     Fourier transform.
    y[1] /= mo2;
    fac=1.0;                             Window function.
    for (j=1;j<mo2;j++) {                Multiply the data by the window function.
        k=2*j+1;
        if (fac) {
```

```
            if ( (fac=(1.0-cnst*j*j)/mo2) < 0.0) fac=0.0;
            y[k]=fac*y[k];
            y[k+1]=fac*y[k+1];
        } else y[k+1]=y[k]=0.0;          Don't do unnecessary multiplies after window function
    }                                        is zero.
    if ( (fac=(1.0-0.25*pts*pts)/mo2) < 0.0) fac=0.0;          Last point.
    y[2] *= fac;
    realft(y,mo2,-1);                    Inverse Fourier transform.
    for (j=1;j<=n;j++)                   Restore the linear trend.
        y[j] += rn1*(y1*(n-j)+yn*(j-1));
}
```

There is a different smoothing technique that we have found useful for data whose error distribution has very broad tails: At each data point, construct a "windowed median" of ordinates, that is the median of that point's ordinate and the ordinates of the $2M$ data points nearest in abscissa, M on each side. Near the edge of the graph, where there are fewer than M points available on one side, instead use more points on the other side, so that your medians are always of $2M$ points. Choose M to taste. The windowed median is not smooth (in the sense of differentiable), but it does fluctuate substantially less than the raw data, and it will often track otherwise noisy trends quite well. It is also genuinely nonparametric, i.e. invariant under reparametrizations of both abscissa and ordinate.

REFERENCES AND FURTHER READING:

Bevington, Philip R. 1969, *Data Reduction and Error Analysis for the Physical Sciences* (New York: McGraw-Hill), §13.1.